Department of Computer Engineering
Academic Term: July-November 2022

Class : B.*E. Computer* **Subject Name :*ML***
Semester : VII **Subject Code : *CSC701***

Practical No:	1
Title.	To Study and implement Linear Regression
Date of Performance.	18-07-2025
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation

Indicator	**Very Poor**	**Poor**	**Average**	**Good**	**Excellent**
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

In [ ]:
```
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
```

In [ ]:
```
file = pd.read_csv('/content/sample_data/Salary_Data.csv')
```

In [ ]:
```
x = file[['YearsExperience']]
y = file['Salary']

print("X and Y : ")
print(x.shape)
print(y.shape)
```

```
X and Y :
(30, 1)
(30,)
```

In [ ]:
```
from sklearn.model_selection import train_test_split
```

In [ ]:
```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_sta

print("\nX_TRAIN and Y_TRAIN : ")
print(x_train.shape)
print(y_train.shape)

print("\nX_TEST and Y_TEST : ")
print(x_test.shape)
print(y_test.shape)
```

```
X_TRAIN and Y_TRAIN :
(24, 1)
(24,)

X_TEST and Y_TEST :
(6, 1)
(6,)
```

In [ ]:
```
from sklearn.linear_model import LinearRegression
```

In [ ]:
```
model = LinearRegression()

model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print("\nModel predict y values: ")
print(y_pred)
```

```
Model predict y values:
[ 40748.96184072 122699.62295594  64961.65717022  63099.14214487
 115249.56285456 107799.50275317]
```

In [ ]:
```
mtp.scatter(x_train, y_train, color = 'red', label = 'Training Data')
mtp.plot(x_train, model.predict(x_train), color = 'blue', label = 'Regression Line'
mtp.title('Salary v/s Experience (Training Set)')
```

```
mtp.xlabel('Years of Experience')
mtp.ylabel('Salary')
mtp.legend()
mtp.show()
```

In [ ]:
```
mtp.scatter(x_test, y_test, color = 'red', label = 'Test Data')
mtp.plot(x_test, model.predict(x_test), color = 'blue', label = 'Regression Line')
mtp.title('Salary v/s Experience (Test Set)')
mtp.xlabel('Years of Experience')
mtp.ylabel('Salary')
mtp.legend()
mtp.show()
```

In [ ]:
```
# import sklearn.metrics as r2_score
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_pred)

print("R2 Score : ")
print(r2)

c = model.intercept_
m = model.coef_

print("\nModel intercept and coef")
print(m, c)
```

```
R2 Score :
0.988169515729126

Model intercept and coef
[9312.57512673] 26780.09915062818
```

In [ ]:
```
accuracy = []

for i in range(100):
  x_train1, x_test1, y_train1, y_test1 = train_test_split(x, y, test_size = 0.8, ra
  model1 = LinearRegression()
  model1.fit(x_train1, y_train1)
  y_pred1 = model.predict(x_test1)
  r2_1 = r2_score(y_test1, y_pred1)
  accuracy.append(r2_1)
```

```
print(accuracy)
print(nm.max(accuracy))
```

```
[0.9536645755326943, 0.9587276669369498, 0.9599226033873139, 0.9474003096085318, 0.9
621063968368476, 0.9604183789115188, 0.9512638404910294, 0.9593236934450713, 0.95864
64746719688, 0.9412976253548547, 0.9596455139722875, 0.9645945039362755, 0.954645182
8306478, 0.9524283843195558, 0.9574349109881829, 0.9616217677370656, 0.9581808041466
211, 0.9583546185222949, 0.9641428142717868, 0.9616813005555789, 0.9443891385389038,
0.9578347285706635, 0.9509390773626878, 0.9627928948960339, 0.9524032048211271, 0.95
67653204443508, 0.9636873613970725, 0.9612603275880329, 0.9512660019542469, 0.962162
4564917671, 0.9526594452373836, 0.9557050174779026, 0.9638773784528749, 0.9497596777
951478, 0.9520178577454619, 0.9609223936735086, 0.9572491005122411, 0.95870004882875
36, 0.9636878446739479, 0.9422185509273002, 0.9501560722560173, 0.9510736292338142,
0.9639970519061158, 0.9662264814449496, 0.9503771843706749, 0.9473614276653719, 0.96
00741369543193, 0.9574501126663743, 0.96173672184421, 0.9600744775743649, 0.96146837
35473679, 0.950494768868645, 0.9579475923300533, 0.9570848179401535, 0.9526204223354
385, 0.9601767789630493, 0.9529857750528089, 0.938427688033362, 0.9566097445611071,
0.9695111211997747, 0.9579177248530238, 0.9529045551559473, 0.9650129321576381, 0.96
07690590975153, 0.9558401079078208, 0.953821050913366, 0.9633530805881929, 0.9525314
94217113, 0.9596164770706567, 0.9542990700558682, 0.9518009398927234, 0.962840894754
048, 0.9612347425144955, 0.9630980566538064, 0.939956073561345, 0.9663083377924078,
0.9506173252358386, 0.9522705290853881, 0.9551200629157734, 0.9534738892946083, 0.95
91085518943221, 0.9572519800510758, 0.9384132231966089, 0.9581215157168369, 0.958166
5113069906, 0.9595025129155997, 0.9549392840822342, 0.9525312590068257, 0.9539815539
857341, 0.965872795774837, 0.9494500804095631, 0.9470786930345976, 0.953711693254650
7, 0.9524522583709272, 0.9409849345516205, 0.9530882303436213, 0.9507159559904956, 0
.9563279411484026, 0.9515636676117039, 0.9534797597250289]
0.9695111211997747
```

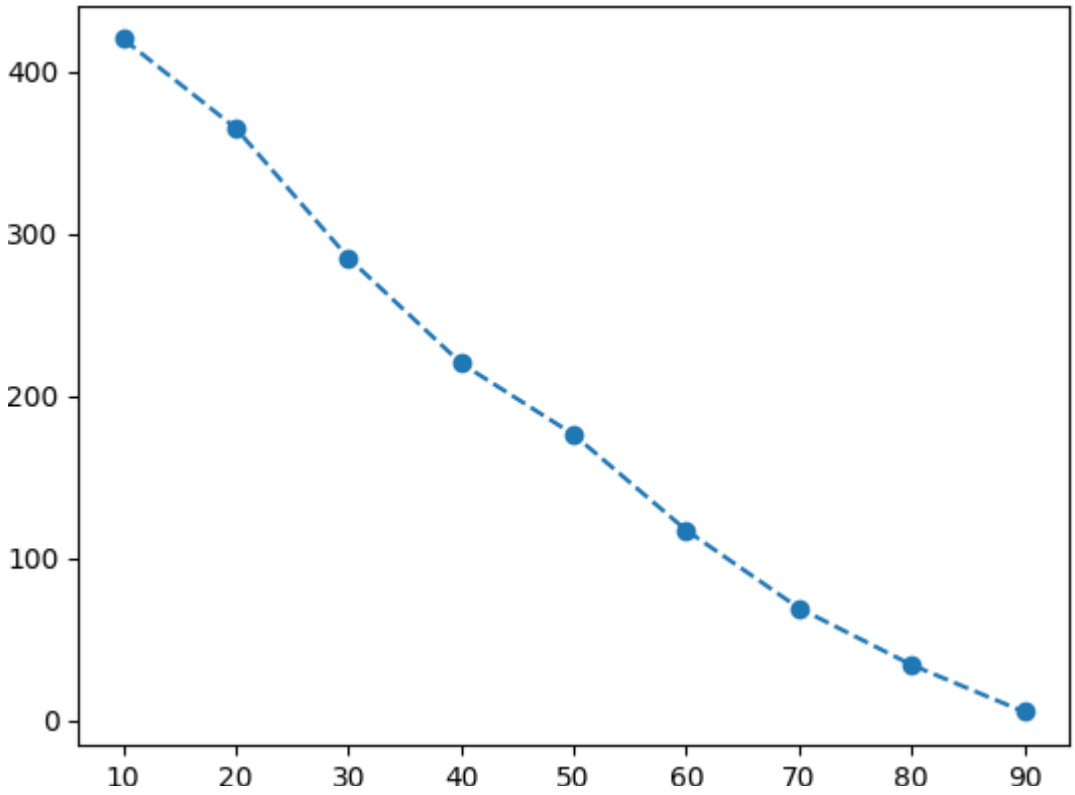
In [ ]:
```
###### POSTLAB 1 ######################################
```

In [ ]:
```
### Q.1)

import numpy as np
import matplotlib.pyplot as mtp
import pandas as pd
import scipy.stats as sp
mtp.rcParams['figure.figsize'] = (20.0, 10.0)

df = {
    'X' : [10,20,30,40,50,60,70,80,90],
    'Y' : [420,365,285,220,176,117,69,34,5]
}

data = pd.DataFrame(df)
print(data.head(9))
```

```
    X    Y
0  10  420
1  20  365
2  30  285
3  40  220
4  50  176
5  60  117
6  70   69
7  80   34
8  90    5
```

In [ ]:
```
X = data['X'].values
Y = data['Y'].values

mtp.plot(X,Y,marker='o',  ls='--')
```

Out[ ]: [<matplotlib.lines.Line2D at 0x7d407c2dfdd0>]



In [ ]:
```
y=np.array(data['Y'], dtype=float)
x=np.array(data['X'],  dtype=float)
slope, intercept, r_value, p_value, std_err =sp.linregress(x,y)
xf = np.linspace(min(x),max(x),100)
yf = (slope*xf)+intercept
print("slope = ",slope,"\n","intercept = ",intercept,"\n",'r = ', r_value**2, '\n',
```

```
slope =  -5.3133333333333335
 intercept =  453.55555555555554
 r =  0.9840369938137091
 p =  1.5050387692160386e-07
 s =  0.2557818184006401
```

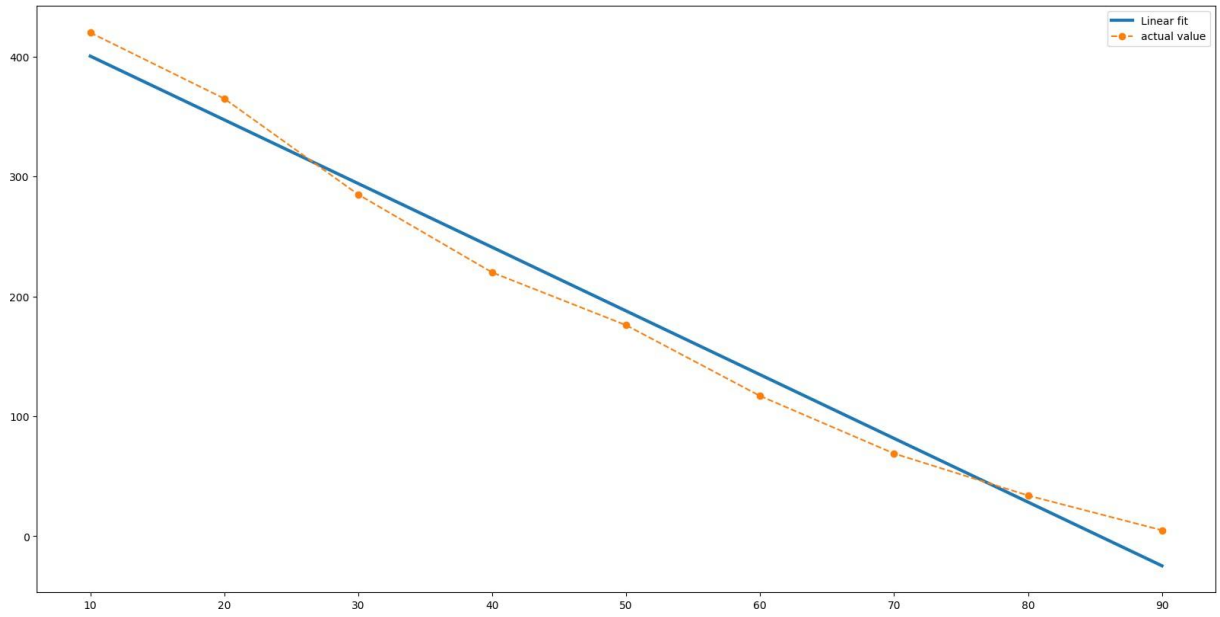
In [ ]:
```
data["expected"] = data["X"] * slope + intercept

data.head(9)
```

Out[ ]:

	X	Y	expected
0	10	420	400.422222
1	20	365	347.288889
2	30	285	294.155556
3	40	220	241.022222
4	50	176	187.888889
5	60	117	134.755556
6	70	69	81.622222
7	80	34	28.488889
8	90	5	-24.644444

In [ ]:
```
f, ax = mtp.subplots(1, 1)
ax.plot(xf, yf,label='Linear fit', lw=3)
ax.plot(X,Y,label="actual value",marker='o', ls='--')
mtp.ylabel('')
ax.legend()
```

Out[ ]: <matplotlib.legend.Legend at 0x7d407ccf8650>



In [ ]:
```
from scipy.stats import t
tinv = lambda p, df: abs(t.ppf(p/2, df))
ts = tinv(0.05, len(X)-2)
```

In [ ]:
```
print(f"slope (95%): {slope:.6f} +/- {ts*std_err:.6f}")
```

```
slope (95%): -5.313333 +/- 0.604828
```

In [ ]:
```
print(f"intercept (95%): {intercept:.6f}"f" +/- {ts*std_err:.6f}")
```

```
intercept (95%): 453.555556 +/- 0.604828
```

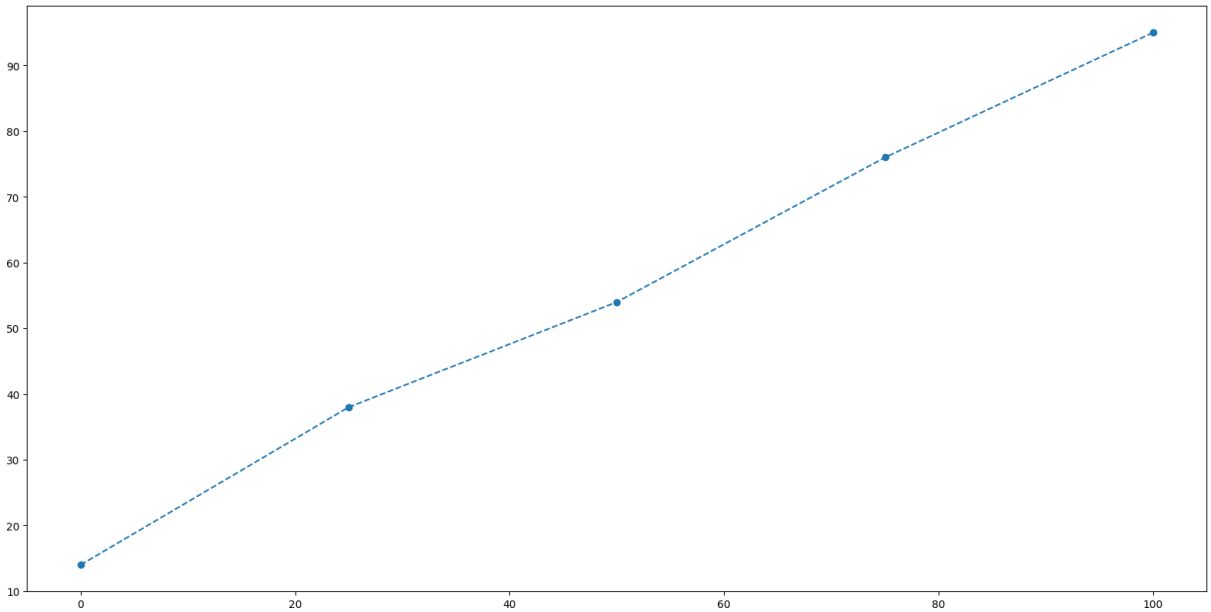
In [ ]:
```
############### Question 2 ###################
```

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import scipy.stats as sp
plt.rcParams['figure.figsize'] = (20.0, 10.0)

df = {
    'X':[0,25,50,75,100],
    'Y':[14,38,54,76,95]
}

data = pd.DataFrame(df)
print(data.head(9))
```

```
     X   Y
0    0  14
1   25  38
2   50  54
3   75  76
4  100  95
```

In [ ]:
```
X = data['X'].values
Y = data['Y'].values


plt.plot(X, Y, marker = 'o', ls = '--')
```

Out[ ]: `[<matplotlib.lines.Line2D at 0x7d407beefdd0>]`



In [ ]:
```
y=np.array(data['Y'], dtype=float)
x=np.array(data['X'],  dtype=float)
slope, intercept, r_value, p_value, std_err =sp.linregress(x,y)
xf = np.linspace(min(x),max(x),100)
yf = (slope*xf)+intercept
```

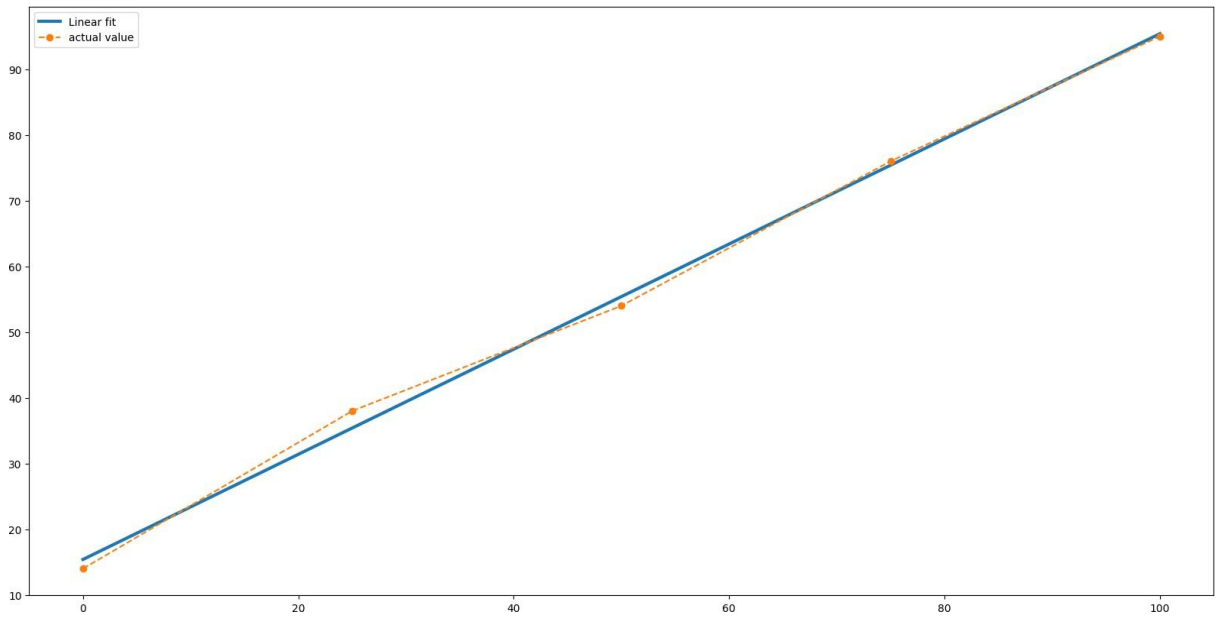
```
print("slope = ",slope,"\n","intercept = ",intercept,"\n",'r = ', r_value**2, '\n',
```

```
slope =  0.8
 intercept =  15.399999999999999
 r =  0.9972078181092939
 p =  6.267128567572262e-05
 s =  0.02444040370643135
```

In [ ]:
```
data["expected"]=data["X"]*slope+intercept
print(data.head())
plt.rcParams['figure.figsize'] = (20.0, 10.0)
f, ax = plt.subplots(1, 1)
ax.plot(xf, yf,label='Linear fit', lw=3)
ax.plot(X,Y,label="actual value",marker='o', ls='--')
plt.ylabel('')
ax.legend()
```

```
     X   Y  expected
0    0  14      15.4
1   25  38      35.4
2   50  54      55.4
3   75  76      75.4
4  100  95      95.4
```

Out[ ]: <matplotlib.legend.Legend at 0x7d407e7e0650>



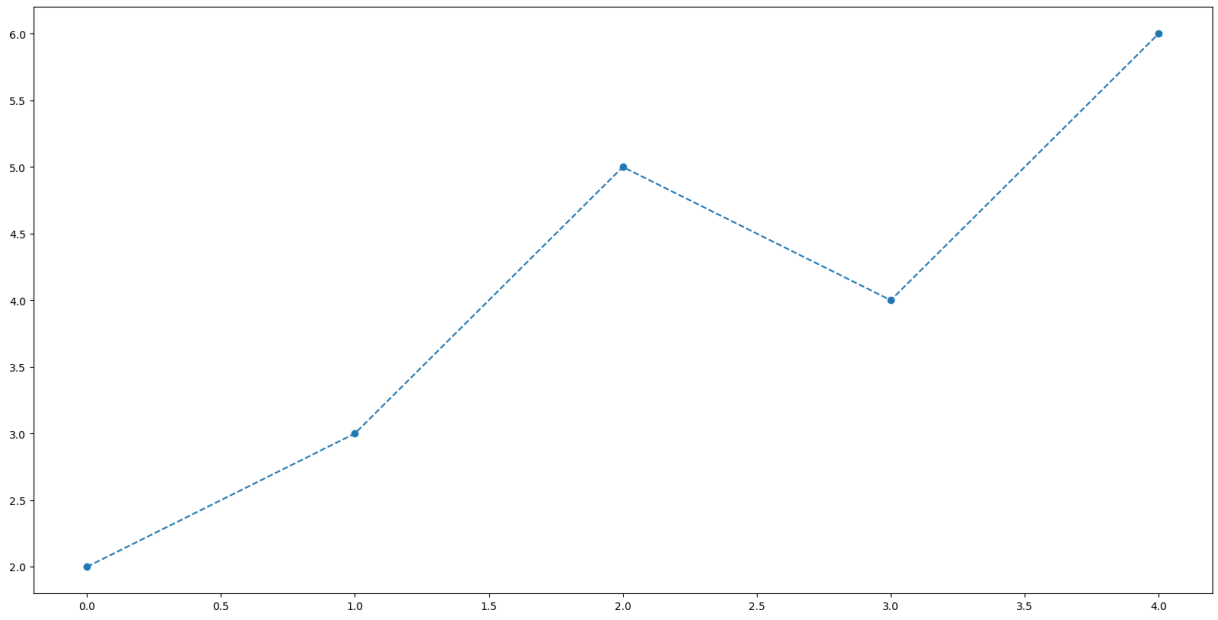
In [ ]:
```
from scipy.stats import t
tinv = lambda p, df: abs(t.ppf(p/2, df))
ts = tinv(0.05, len(X)-2)
print(f"slope (95%): {slope:.6f} +/- {ts*std_err:.6f}")
print(f"intercept (95%): {intercept:.6f}"f" +/- {ts*std_err:.6f}")
```

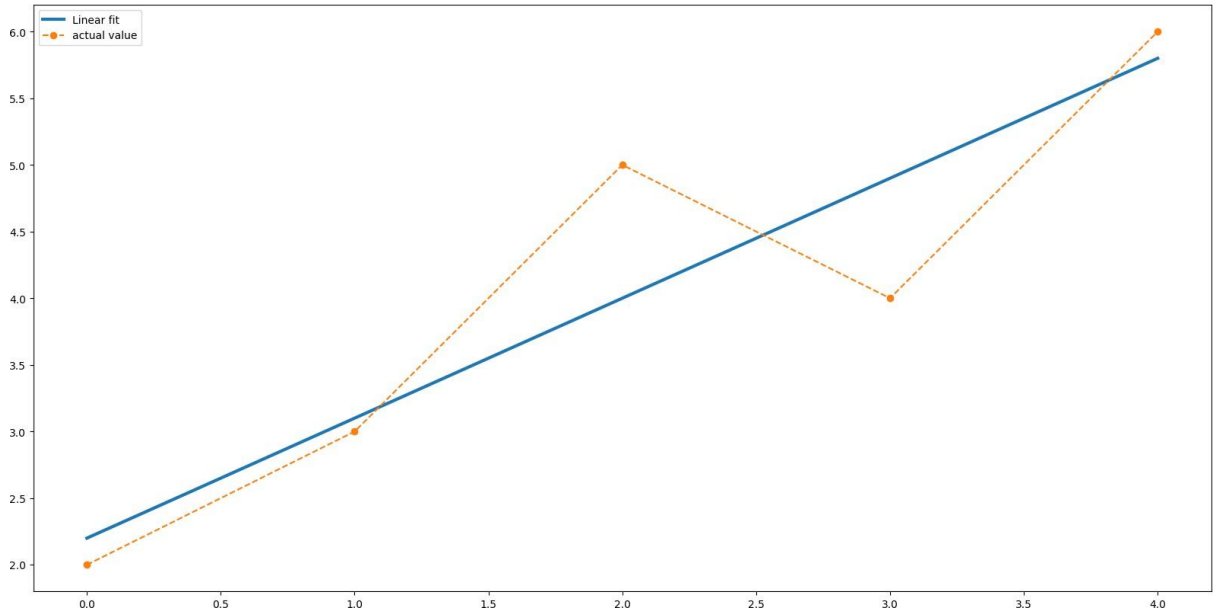
```
slope (95%): 0.800000 +/- 0.077780
intercept (95%): 15.400000 +/- 0.077780
```

In [ ]:
```
########### Question 3 ###################
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import scipy.stats as sp
plt.rcParams['figure.figsize'] = (20.0, 10.0)

df = {
      'X':[0,1,2,3,4],
      'Y':[2,3,5,4,6]
    }

data = pd.DataFrame(df)
print (data.head(9))

X = data['X'].values
Y = data['Y'].values
plt.plot(X,Y,marker='o',  ls='--')

y=np.array(data['Y'], dtype=float)
x=np.array(data['X'],  dtype=float)
slope, intercept, r_value, p_value, std_err =sp.linregress(x,y)
xf = np.linspace(min(x),max(x),100)

yf = (slope*xf)+intercept
print("slope=",slope,"\n","intercept=",intercept,"\n",'r = ', r_value**2, '\n', 'p
print(f"\nequation of line: y={slope:.2f}x+{intercept:.2f}")

data["expected"]=data["X"]*slope+intercept
print(data.head())
plt.rcParams['figure.figsize'] = (20.0, 10.0)
f, ax = plt.subplots(1, 1)
ax.plot(xf, yf,label='Linear fit', lw=3)
ax.plot(X,Y,label="actual value",marker='o', ls='--')
plt.ylabel('')
ax.legend()

from scipy.stats import t
tinv = lambda p, df: abs(t.ppf(p/2, df))
pred = lambda x: slope*x+intercept
ts = tinv(0.05, len(X)-2)
print(f"slope (95%): {slope:.6f} +/- {ts*std_err:.6f}")
print(f"intercept (95%): {intercept:.6f}"f" +/- {ts*std_err:.6f}")
print("value of y at x=10=",pred(10))
```

```
   X  Y
0  0  2
1  1  3
2  2  5
3  3  4
4  4  6
slope= 0.9
 intercept= 2.2
 r =  0.81
 p =  0.03738607346849863
 s =  0.25166114784235827

equation of line: y=0.90x+2.20
   X  Y  expected
0  0  2       2.2
1  1  3       3.1
2  2  5       4.0
3  3  4       4.9
4  4  6       5.8
slope (95%): 0.900000 +/- 0.800898
intercept (95%): 2.200000 +/- 0.800898
value of y at x=10= 11.2
```

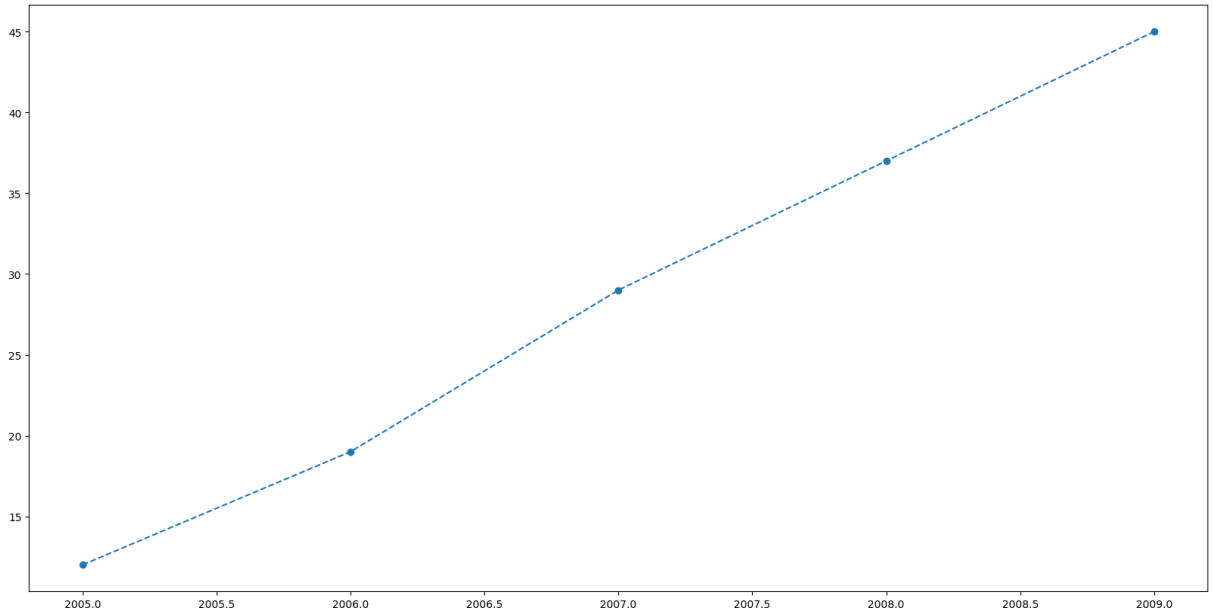


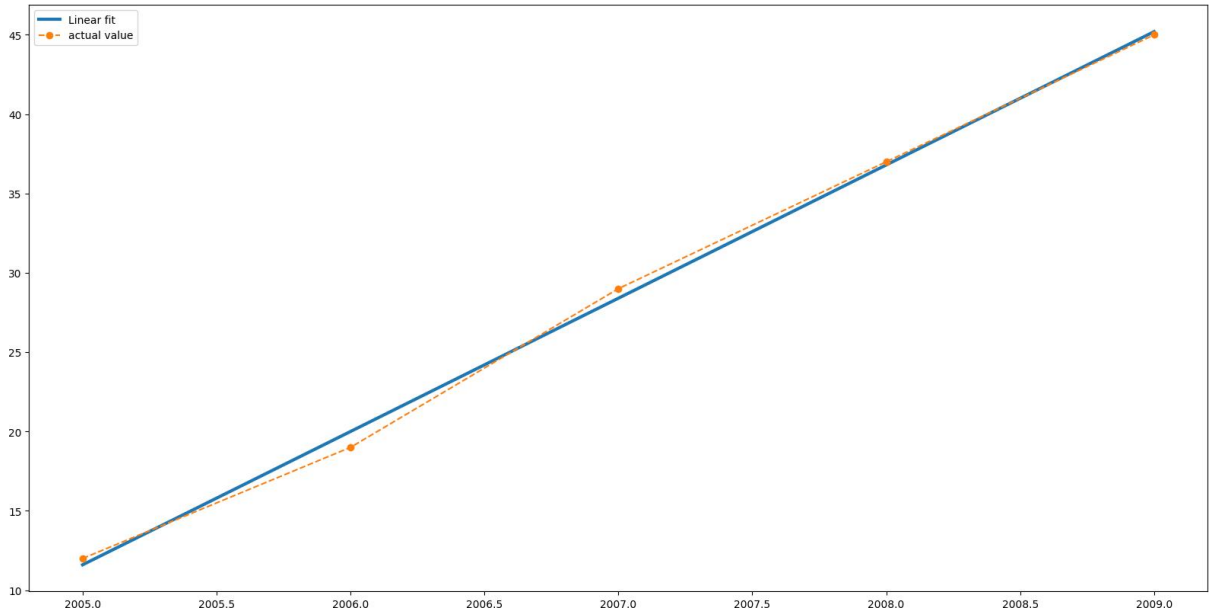


In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as sp
plt.rcParams['figure.figsize'] = (20.0, 10.0)


df = {'X':[2005,2006,2007,2008,2009],
'Y':[12,19,29,37,45]}


data = pd.DataFrame(df)
print (data.head(9))


X = data['X'].values
Y = data['Y'].values
plt.plot(X,Y,marker='o',  ls='--')


y=np.array(data['Y'], dtype=float)
x=np.array(data['X'],  dtype=float)
slope, intercept, r_value, p_value, std_err =sp.linregress(x,y)
xf = np.linspace(min(x),max(x),100)


yf = (slope*xf)+intercept
print("slope=",slope,"\n","intercept=",intercept,"\n",'r = ', r_value**2, '\n', 'p
print(f"\nequation of line: y={slope:.2f}x+{intercept:.2f}")

data["expected"]=data["X"]*slope+intercept
print(data.head())
plt.rcParams['figure.figsize'] = (20.0, 10.0)
f, ax = plt.subplots(1, 1)
ax.plot(xf, yf,label='Linear fit', lw=3)
ax.plot(X,Y,label="actual value",marker='o', ls='--')
plt.ylabel('')
```

```
ax.legend()


from scipy.stats import t
tinv = lambda p, df: abs(t.ppf(p/2, df))
pred = lambda x: slope*x+intercept
ts = tinv(0.05, len(X)-2)
print(f"slope (95%): {slope:.6f} +/- {ts*std_err:.6f}")
print(f"intercept (95%): {intercept:.6f}"f" +/- {ts*std_err:.6f}")
print("value of y at x=2012=",pred(2012))
```

```
      X   Y
0  2005  12
1  2006  19
2  2007  29
3  2008  37
4  2009  45
slope= 8.4
 intercept= -16830.399999999998
 r =  0.9977375565610856
 p =  4.570360814080035e-05
 s =  0.23094010767586692

equation of line: y=8.40x+-16830.40
      X   Y  expected
0  2005  12      11.6
1  2006  19      20.0
2  2007  29      28.4
3  2008  37      36.8
4  2009  45      45.2
slope (95%): 8.400000 +/- 0.734954
intercept (95%): -16830.400000 +/- 0.734954
value of y at x=2012= 70.40000000000146
```

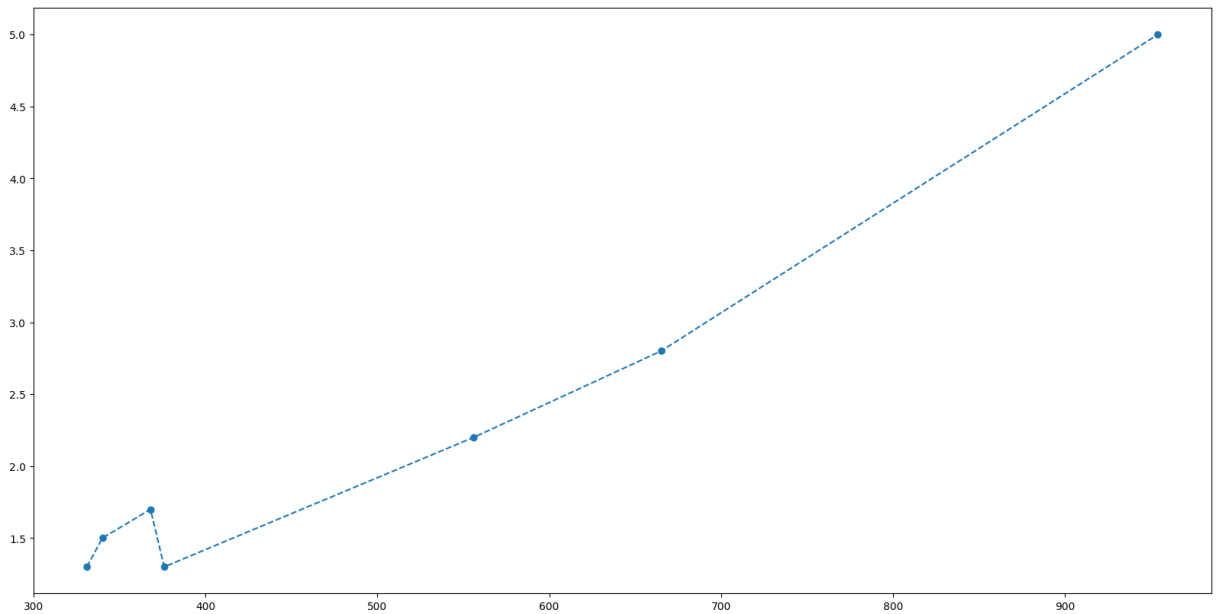


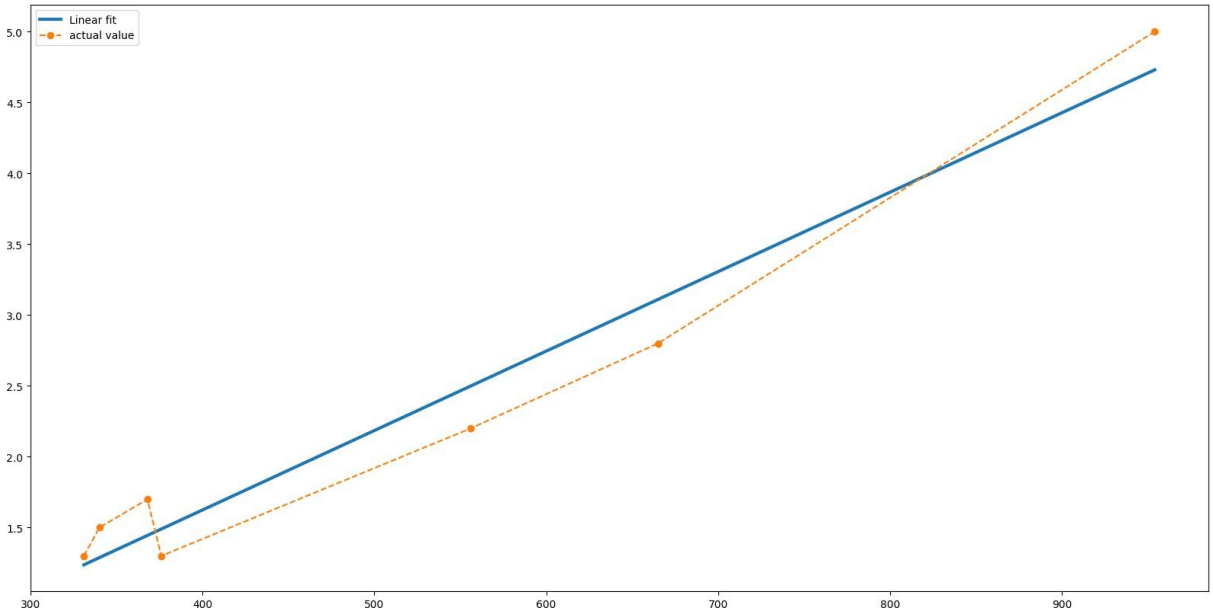


In [ ]:
```
############## Question 5 ################
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as sp
plt.rcParams['figure.figsize'] = (20.0, 10.0)


df = {'X':[368,340,665,954,331,556,376],
'Y':[1.7,1.5,2.8,5,1.3,2.2,1.3]}


data = pd.DataFrame(df)
data=data.sort_values('X')
print (data.head(9))


X = data['X'].values
Y = data['Y'].values
plt.plot(X,Y,marker='o',  ls='--')


y=np.array(data['Y'], dtype=float)
x=np.array(data['X'],  dtype=float)
slope, intercept, r_value, p_value, std_err =sp.linregress(x,y)
xf = np.linspace(min(x),max(x),100)


yf = (slope*xf)+intercept
print("slope=",slope,"\n","intercept=",intercept,"\n",'r = ', r_value**2, '\n', 'p
print(f"\nequation of line: y={slope:.2f}x+{intercept:.2f}")

data["expected"]=data["X"]*slope+intercept
print(data.head())
plt.rcParams['figure.figsize'] = (20.0, 10.0)
f, ax = plt.subplots(1, 1)
ax.plot(xf, yf,label='Linear fit', lw=3)
```

```
ax.plot(X,Y,label="actual value",marker='o', ls='--')
plt.ylabel('')
ax.legend()


from scipy.stats import t
tinv = lambda p, df: abs(t.ppf(p/2, df))
pred = lambda x: slope*x+intercept
ts = tinv(0.05, len(X)-2)
print(f"slope (95%): {slope:.6f} +/- {ts*std_err:.6f}")
print(f"intercept (95%): {intercept:.6f}"f" +/- {ts*std_err:.6f}")
print("value of y at x=8=",pred(8))
```

```
     X    Y
4   331  1.3
1   340  1.5
0   368  1.7
6   376  1.3
5   556  2.2
2   665  2.8
3   954  5.0
slope= 0.005606157184993036
 intercept= -0.6180148991607144
 r =  0.9612629035488399
 p =  0.00010169537218360504
 s =  0.0005032951082414453

equation of line: y=0.01x+-0.62
     X    Y  expected
4   331  1.3  1.237623
1   340  1.5  1.288079
0   368  1.7  1.445051
6   376  1.3  1.489900
5   556  2.2  2.499008
slope (95%): 0.005606 +/- 0.001294
intercept (95%): -0.618015 +/- 0.001294
value of y at x=8= -0.5731656416807701
```




Linear fit
actual value
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
300
400
500
600
700
800
900

Department of Computer Engineering
Academic Term: July-November 2022

Class : B.*E. Computer* **Subject Name :*ML***
Semester : VII **Subject Code : *CSC701***

Practical No:	2
Title.	To Study and implement Multivariate Linear Regression
Date of Performance.	25-07-2025
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation

Indicator	**Very Poor**	**Poor**	**Average**	**Good**	**Excellent**
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error


data_set= load_diabetes()


df = pd.DataFrame(data_set.data)


df
```

	0	1	2	3	4	5	6	7	8	9
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641
...	...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193	0.007207
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114	0.044485
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883	0.015491
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529	-0.025930
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222	0.003064

442 rows × 10 columns

```
n_df = pd.DataFrame({f'feature{i+1}': df[i] for i in range(10)})
display(n_df)
```

	feature1	feature2	feature3	feature4	feature5	feature6	feature7	feature8	feature9	feature10
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641
...	...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193	0.007207
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114	0.044485
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883	0.015491
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529	-0.025930
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222	0.003064

442 rows × 10 columns

```
n_df.rename(columns={'feature10': 'target'}, inplace=True)


n_df
```

	feature1	feature2	feature3	feature4	feature5	feature6	feature7	feature8	feature9	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641
...	...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193	0.007207
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114	0.044485
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883	0.015491
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529	-0.025930
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222	0.003064

442 rows × 10 columns

```
n_df.shape
```

```
(442, 10)
```

```
X = n_df[[f"feature{i}" for i in range(1, 10)]]
Y = n_df['target']
```

X

	feature1	feature2	feature3	feature4	feature5	feature6	feature7	feature8	feature9
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988
...	...	...	...	...	...	...	...	...	...
437	0.041708	0.050680	0.019662	0.059744	-0.005697	-0.002566	-0.028674	-0.002592	0.031193
438	-0.005515	0.050680	-0.015906	-0.067642	0.049341	0.079165	-0.028674	0.034309	-0.018114
439	0.041708	0.050680	-0.015906	0.017293	-0.037344	-0.013840	-0.024993	-0.011080	-0.046883
440	-0.045472	-0.044642	0.039062	0.001215	0.016318	0.015283	-0.028674	0.026560	0.044529
441	-0.045472	-0.044642	-0.073030	-0.081413	0.083740	0.027809	0.173816	-0.039493	-0.004222

442 rows × 9 columns

Y

	target
0	-0.017646
1	-0.092204
2	-0.025930
3	-0.009362
4	-0.046641
...	...
437	0.007207
438	0.044485
439	0.015491
440	-0.025930
441	0.003064

442 rows × 1 columns

dtype: float64

```
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42)


model = LinearRegression()


model.fit(X_train,y_train)
```

```
LinearRegression
LinearRegression()
```

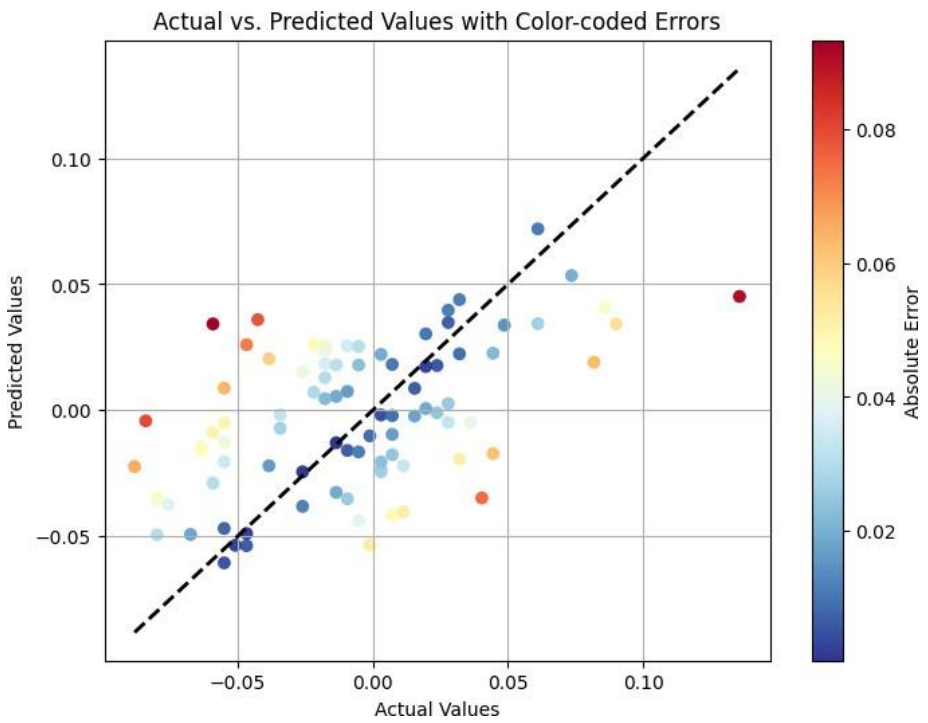
```
predictions = model.predict(X_test)


predictions
```

```
array([ 0.01795242,  0.00714908,  0.02570788,  0.07197301,  0.00749053,
       -0.00957196,  0.04389301,  0.03435451, -0.02439529, -0.01014245,
       -0.02200251, -0.00419319, -0.06050463,  0.03423753, -0.01715733,
       -0.01930115,  0.04074458,  0.05346341, -0.00226367,  0.03969703,
        0.00552473, -0.0146184 , -0.04140273,  0.03030806, -0.01592567,
        0.00870107,  0.01847317,  0.01795839, -0.05369374, -0.00098385,
        0.00069896, -0.0380798 , -0.00184815,  0.02238674,  0.0177745 ,
        0.02550624, -0.00476394,  0.00263898,  0.01907617, -0.03739968,
       -0.03476065, -0.00211286,  0.00877726,  0.02261898,  0.01819353,
       -0.03476505, -0.0533734 , -0.02339817, -0.04889181, -0.00488207,
       -0.02440359, -0.04935076, -0.00871194, -0.02043729,  0.03596397,
       -0.01583155, -0.02205483,  0.02369848, -0.02041138, -0.04381903,
        0.0221781 ,  0.00461399, -0.01650421, -0.0047823 , -0.01288328,
        0.03425578,  0.02530696,  0.0204953 , -0.0355862 , -0.00178393,
        0.0129312 ,  0.03482119,  0.04515843, -0.00713976, -0.02884561,
        0.02600916,  0.03376684,  0.01543325,  0.01740668,  0.02593953,
       -0.03254013, -0.02231277, -0.04684546, -0.03509398, -0.01761152,
       -0.04027806, -0.04923399, -0.05353426, -0.0125186 ])
```

```
from sklearn.metrics import r2_score
from sklearn.metrics import accuracy_score
print(
  'MSE : ', mean_squared_error(y_test, predictions))
print(
  'MAE : ', mean_absolute_error(y_test, predictions))
print('R-S: ', r2_score(y_test, predictions))
```

```
MSE :  0.0014397511613178668
     MAE :  0.031211788744898387
    R-S:  0.24545282143100922
```

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, predictions, c=np.abs(y_test - predictions), cmap='RdYlBu_r')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs. Predicted Values with Color-coded Errors")
plt.colorbar(label="Absolute Error")
plt.grid(True)
plt.show()
```




```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
```

```
housing_df = pd.DataFrame(housing.data, columns=housing.feature_names)
housing_df['target'] = housing.target
```

```
housing_df
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	target
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422
...	...	...	...	...	...	...	...	...	...
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	-121.09	0.781
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	-121.21	0.771
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	-121.22	0.923
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	-121.32	0.847
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	-121.24	0.894

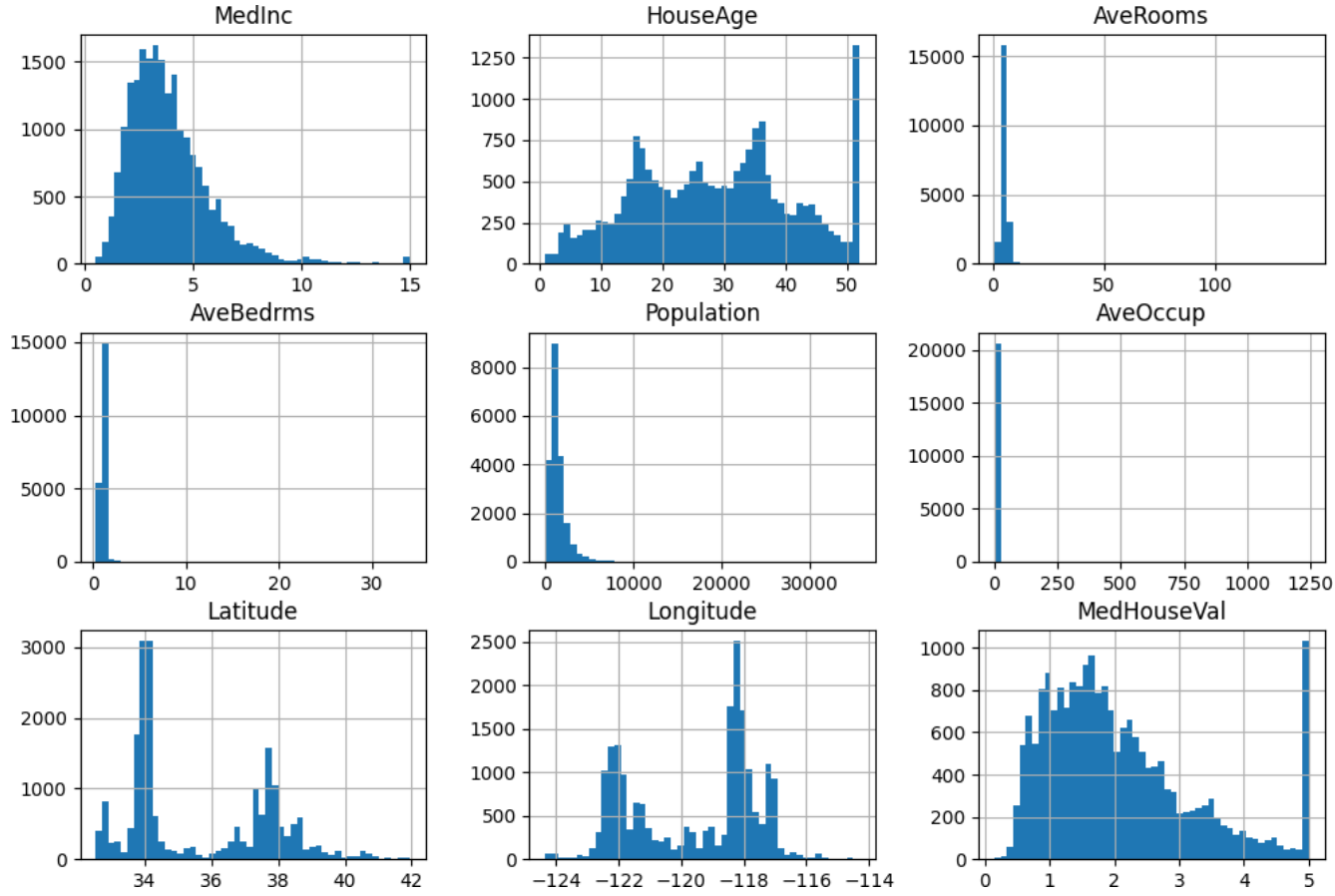
20640 rows × 9 columns

```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix
from sklearn.metrics import r2_score
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

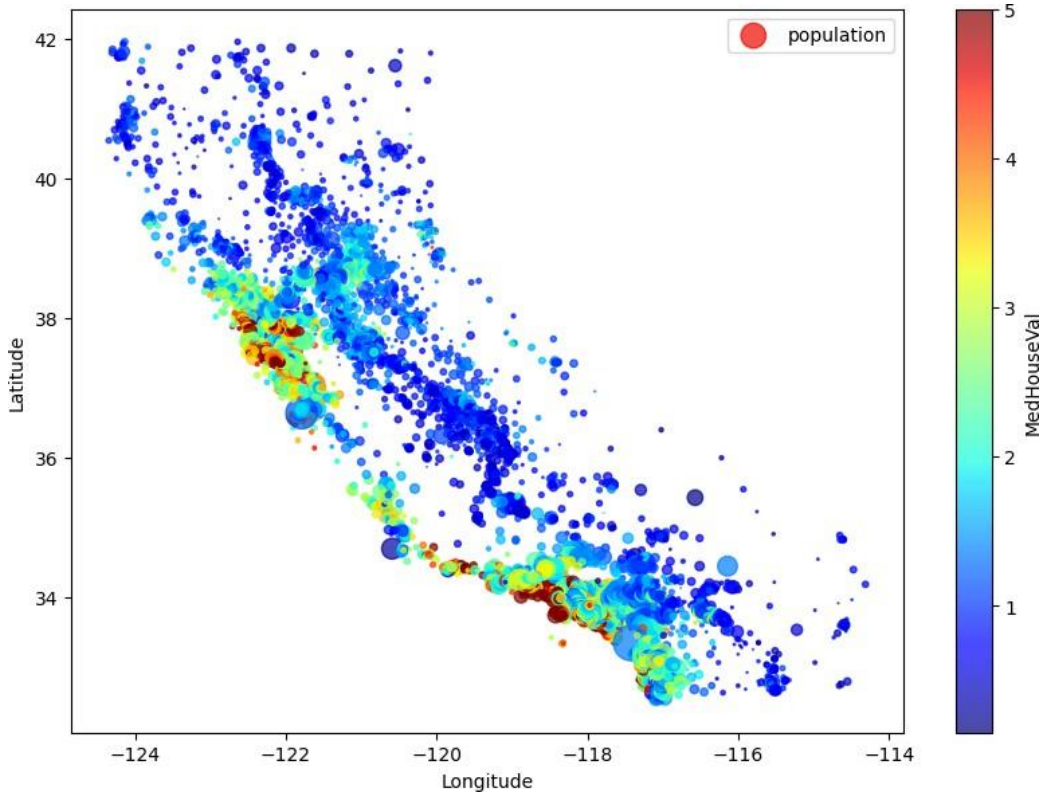
```
housing = fetch_california_housing(as_frame=True)
housing = housing.frame
housing.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

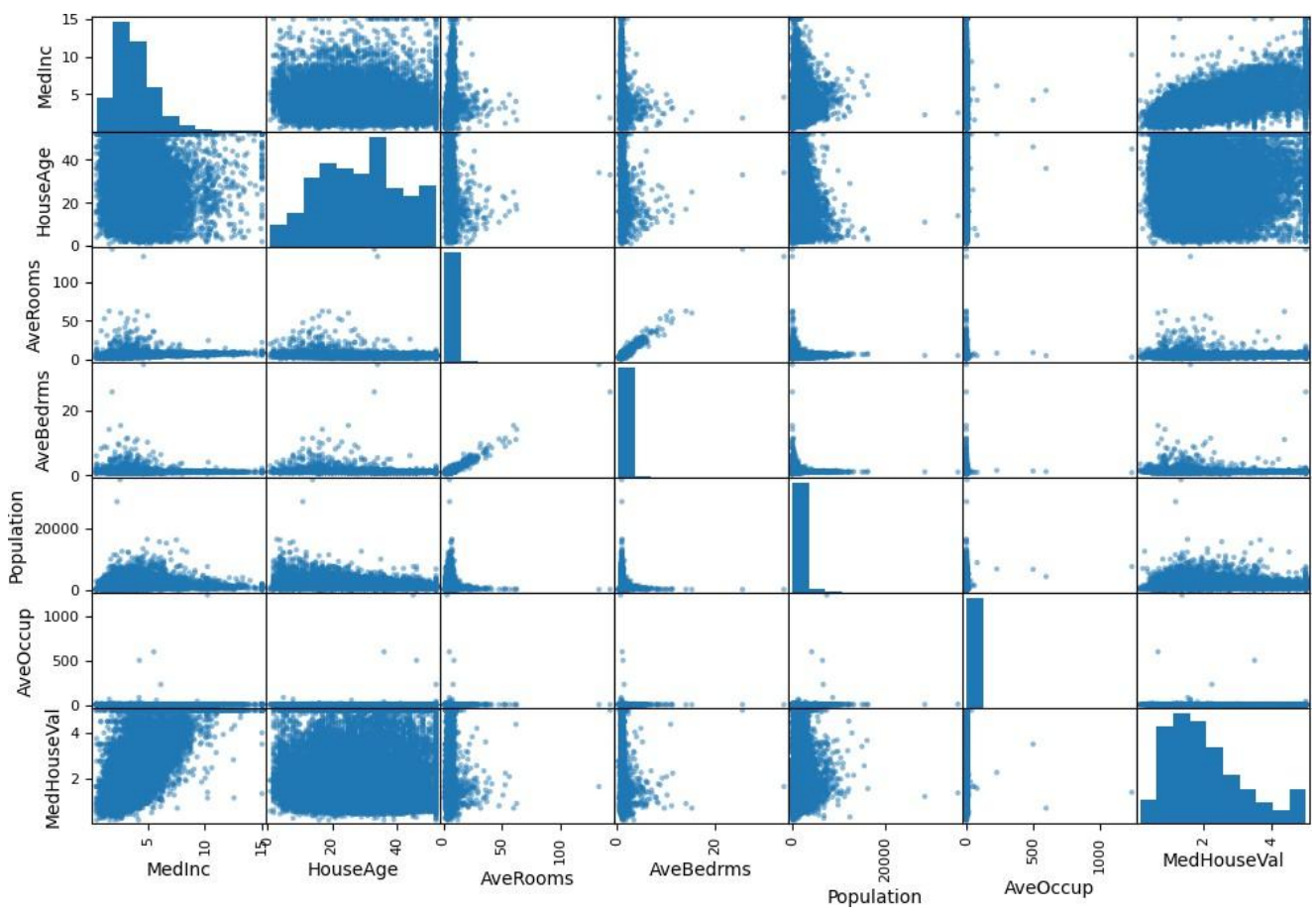
```
housing.hist(bins=50, figsize=(12,8))
plt.show()
```




```
housing.plot(kind="scatter", x="Longitude",y="Latitude", c="MedHouseVal", cmap="jet", colorbar=True, legend=True, sharex=False, figsize
plt.show()
```




```
attributes = ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup','MedHouseVal']
scatter_matrix(housing[attributes], figsize=(12,8))
plt.show()
```




```
corr = housing.corr()
corr['MedHouseVal'].sort_values(ascending=True)
```

	MedHouseVal
Latitude	-0.144160
AveBedrms	-0.046701
Longitude	-0.045967
Population	-0.024650
AveOccup	-0.023737
HouseAge	0.105623
AveRooms	0.151948
MedInc	0.688075
MedHouseVal	1.000000

Q1 Is a multivariate regression the same as linear regression?

No, multivariate regression is not the same as linear regression. Linear regression involves one independent and one dependent variable. In contrast, multivariate regression uses two or more independent variables, and sometimes multiple dependent variables, to study more complex relationships.

Q2 What is a multivariate regression with example?

Multivariate regression is a method used to model the relationship between multiple inputs and multiple outputs. For example, it can be used to predict both blood pressure and cholesterol level based on variables like age, weight, and diet. It's helpful in analyzing systems with multiple influencing

Q3 Why do we use multivariate regression models?

We use multivariate regression to understand how several variables affect one or more outcomes. It allows researchers to model real-world situations more accurately and uncover relationships that wouldn't be visible with simpler models. It's especially useful when outcomes are influenced by many factors.

Q4 What are the advantages of multivariate regression?

It offers improved prediction accuracy and reveals complex variable interactions. It allows control over confounding variables and helps researchers make more reliable and generalizable conclusions. It's also efficient, as it analyzes multiple variables in a single model.

Q5 Is multivariable regression linear regression?

Yes, multivariable regression is a type of linear regression that involves more than one independent variable predicting a single dependent variable. Although the number of predictors increases, the relationship between variables remains linear. It's an extension of simple linear regression.

Q6 What is the difference between univariate and multivariate regression?

Univariate regression involves only one dependent variable and usually one independent variable.

Multivariate regression, on the other hand, can include multiple independent and/or dependent variables. It provides a deeper understanding of how different factors influence outcomes together.

Q7 Where is multivariate analysis put to use?

Multivariate analysis is used in many fields like marketing, education, healthcare, and social sciences. It helps in understanding how multiple factors work together to influence outcomes. For example, in marketing, it can analyze customer behavior based on age, income, and preferences.

Q8 Is multivariate analysis qualitative or quantitative?

Multivariate analysis is primarily quantitative, as it often deals with numerical data. However, it can also be applied to qualitative or categorical data using techniques like logistic regression. It's flexible enough to handle both types depending on the method used.

Department of Computer Engineering
Academic Term: July-November 2022

Class : B.*E. Computer* **Subject Name :*ML***
Semester : VII **Subject Code : *CSC701***

Practical No:	3
Title.	To Study and implement Logistic regression
Date of Performance.	01-08-2025
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation

Indicator	**Very Poor**	**Poor**	**Average**	**Good**	**Excellent**
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

In [ ]:
```
import numpy as np
import pandas as pd

from sklearn import preprocessing
import matplotlib.pyplot as plt
plt.rc("font", size=14)
import seaborn as sns
sns.set(style="white") #white background style for seaborn plots
sns.set(style="whitegrid", color_codes=True)

import warnings
warnings.simplefilter(action='ignore')
```

New Section

In [ ]:
```
train_df = pd.read_csv("/content/titanic.csv")

# Read CSV test data file into DataFrame
test_df = pd.read_csv("/content/titanic.csv")

# preview train data
train_df.head()
```

Out[ ]:

	passengerid	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	1	1	Allen, Miss. Elisabeth Walton	female	29.0000	0	0	24160	211.3375	B5	S
1	2	1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.5500	C22 C26	S
2	3	1	0	Allison, Miss. Helen Loraine	female	2.0000	1	2	113781	151.5500	C22 C26	S
3	4	1	0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1	2	113781	151.5500	C22 C26	S
4	5	1	0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1	2	113781	151.5500	C22 C26	S

In [ ]:
```
print('The number of samples into the train data is {}.'.format(train_df.shape[0]))
```

```
The number of samples into the train data is 1309.
```

In [ ]:
```
test_df.head()
```

Out[ ]:

	passengerid	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1	1	1	Allen, Miss. Elisabeth Walton	female	29.0000	0	0	24160	211.3375	B5	S
1	2	1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.5500	C22 C26	S
2	3	1	0	Allison, Miss. Helen Loraine	female	2.0000	1	2	113781	151.5500	C22 C26	S
3	4	1	0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1	2	113781	151.5500	C22 C26	S
4	5	1	0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1	2	113781	151.5500	C22 C26	S

In [ ]:
```
print('The number of samples into the test data is {}.'.format(test_df.shape[0]))
```

```
The number of samples into the test data is 1309.
```

In [ ]:
```
train_df.isnull().sum()
```

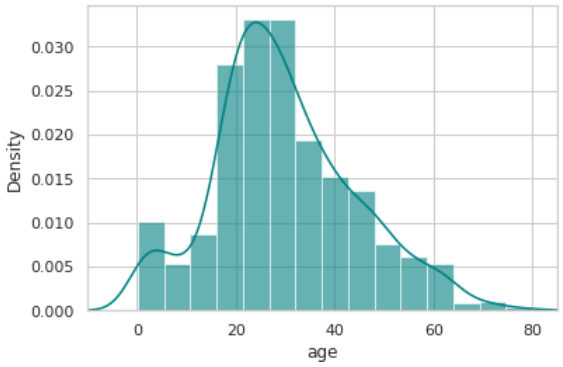
Out[ ]:
```
passengerid       0
pclass            0
survived          0
name              0
sex               0
age             263
sibsp             0
parch             0
ticket            0
fare              1
cabin          1014
embarked          2
dtype: int64
```

In [ ]:
```
print('Percent of missing "Age" records is %.2f%%' %((train_df['age'].isnull().sum()/train_df.shape[0])*100))
```

```
Percent of missing "Age" records is 20.09%
```

In [ ]:
```
ax = train_df["age"].hist(bins=15, density=True, stacked=True, color='teal', alpha=0.6)
train_df["age"].plot(kind='density', color='teal')
ax.set(xlabel='age')
```

```
plt.xlim(-10,85)
plt.show()
```



In [ ]:
```
print('The mean of "Age" is %.2f' %(train_df["age"].mean(skipna=True)))
# median age
print('The median of "Age" is %.2f' %(train_df["age"].median(skipna=True)))
```

```
The mean of "Age" is 29.88
The median of "Age" is 28.00
```

In [ ]:
```
print('Percent of missing "Cabin" records is %.2f%%' %((train_df['cabin'].isnull().sum()/train_df.shape[0])*100))
```

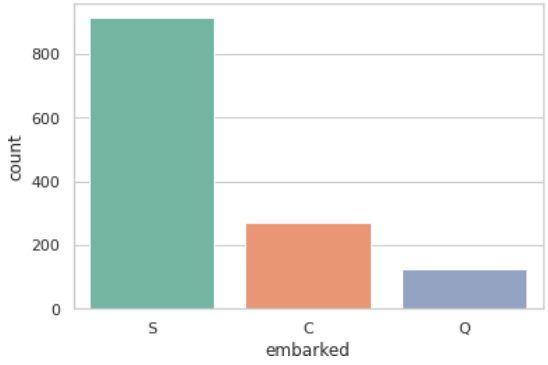
```
Percent of missing "Cabin" records is 77.46%
```

In [ ]:
```
print('Percent of missing "Embarked" records is %.2f%%' %((train_df['embarked'].isnull().sum()/train_df.shape[0])*100))
```

```
Percent of missing "Embarked" records is 0.15%
```

In [ ]:
```
print('Boarded passengers grouped by port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton):')
print(train_df['embarked'].value_counts())
sns.countplot(x='embarked', data=train_df, palette='Set2')
plt.show()
```

```
Boarded passengers grouped by port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton):
S    914
C    270
Q    123
Name: embarked, dtype: int64
```



In [ ]:
```
print('The most common boarding port of embarkation is %s.' %train_df['embarked'].value_counts().idxmax())
```

```
The most common boarding port of embarkation is S.
```

In [ ]:
```
train_data = train_df.copy()
train_data["age"].fillna(train_df["age"].median(skipna=True), inplace=True)
train_data["embarked"].fillna(train_df['embarked'].value_counts().idxmax(), inplace=True)
train_data.drop('cabin', axis=1, inplace=True)
```

In [ ]:
```
train_data.isnull().sum()
```

Out[ ]:
```
passengerid    0
pclass         0
survived       0
name           0
sex            0
age            0
sibsp          0
parch          0
ticket         0
fare           1
embarked       0
dtype: int64
```

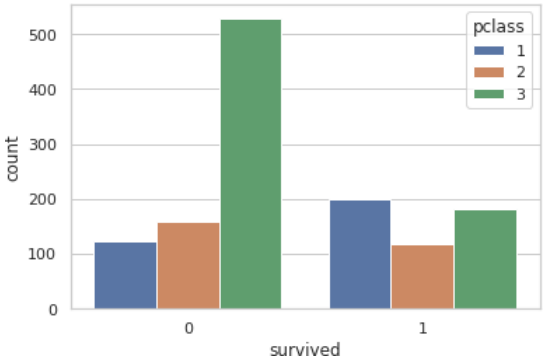
In [ ]:
```
train_data.head()
```

Out[ ]:

	passengerid	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	embarked
0	1	1	1	Allen, Miss. Elisabeth Walton	female	29.0000	0	0	24160	211.3375	S
1	2	1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.5500	S
2	3	1	0	Allison, Miss. Helen Loraine	female	2.0000	1	2	113781	151.5500	S
3	4	1	0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1	2	113781	151.5500	S
4	5	1	0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1	2	113781	151.5500	S

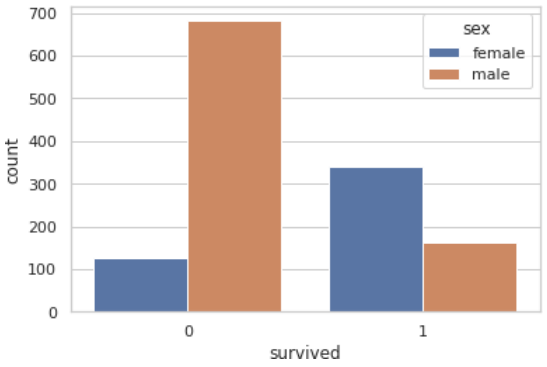
In [ ]:
```
sns.countplot(x='survived', hue='pclass', data=train_df)
```

Out[ ]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7ffac6809a10>
```



In [ ]:
```
sns.countplot(x='survived', hue='sex', data=train_df)
```

Out[ ]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7ffac67084d0>
```



In [ ]:
```
def add_age(cols):
    Age = cols[0]
    Pclass = cols[1]
    if pd.isnull(Age):
        return int(train_df[train_df["pclass"] == Pclass]["age"].mean())
    else:
        return Age
```

In [ ]:
```
train_df["age"] = train_df[["age", "pclass"]].apply(add_age,axis=1)
```

In [ ]:
```
train_df.drop("cabin",inplace=True,axis=1)
```

In [ ]:
```
train_df.dropna(inplace=True)
```

In [ ]:
```
pd.get_dummies(train_df["sex"])
```

Out[ ]:

	female	male
0	1	0
1	0	1
2	1	0
3	0	1
4	1	0
...	...	...
1304	1	0
1305	1	0
1306	0	1
1307	0	1
1308	0	1

1306 rows × 2 columns

In [ ]:
```
sex = pd.get_dummies(train_df["sex"],drop_first=True)
```

In [ ]:
```
embarked = pd.get_dummies(train_df["embarked"],drop_first=True)
pclass = pd.get_dummies(train_df["pclass"],drop_first=True)
```

In [ ]:
```
train = pd.concat([train_df,pclass,sex,embarked],axis=1)
```

In [ ]:
```
train.drop(["passengerid","pclass","name","sex","ticket","embarked"],axis=1,inplace=True)
```

In [ ]:
```
X = train.drop("survived",axis=1)
y = train["survived"]
```

In [ ]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 101)
```

In [ ]:
```
from sklearn.linear_model import LogisticRegression
logmodel = LogisticRegression()
logmodel.fit(X_train,y_train)
```

Out[ ]:
```
LogisticRegression()
```

In [ ]:
```
predictions = logmodel.predict(X_test)
from sklearn.metrics import classification_report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.84      0.86      0.85       247
           1       0.75      0.73      0.74       145

    accuracy                           0.81       392
   macro avg       0.80      0.79      0.80       392
weighted avg       0.81      0.81      0.81       392
```

In [ ]:
```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, predictions)
```

Out[ ]:
```
array([[212,  35],
       [ 39, 106]])
```

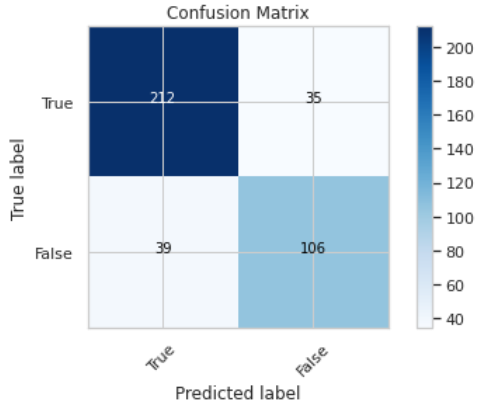
In [ ]:
```
%matplotlib inline
from sklearn.metrics import confusion_matrix
import itertools
import matplotlib.pyplot as plt
```

In [ ]:
```
def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
        print("Normalized confusion matrix")
```

```
    else:
        print('Confusion matrix, without normalization')

    print(cm)

    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, cm[i, j],
             horizontalalignment="center",
             color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
```

In [ ]:
```
cm = confusion_matrix(y_true=y_test, y_pred=predictions)
cm_plot_labels = ["True","False"]
plot_confusion_matrix(cm=cm, classes=cm_plot_labels, title='Confusion Matrix')
```

```
Confusion matrix, without normalization
[[212  35]
 [ 39 106]]
```



Q What is a logistic function?
What is a logistic function's range of values?

Ans A logistic function is a type of sigmoid curve that models growth which starts exponentially but then levels off as it approaches a maximum limit.
The values of a logistic function will range from 0 to 1. The importance of x will vary from -infinity to tinfinity.

Q What makes logistic regression so well-liked?
Logistic regression is famous because it can convert the values of logits (log odds), which can range from
-infinity to tinfinity to a range between 0 and 1. As logistic function outputs the probability of occurrence of an event, it can be applied to many real-life scenarios. It is for this reason that the logistic regression model is prevalent.

Q How can a logistic regression model's probability be stated as a conditional probability?

Ans p(discrete value of target variable $X_1, X_2, X_3 \ldots X_n$). it is the probability of the target variable taking up a discrete value (either 0 or 1 in case of binary classification problems) when the values of independent variables are given. For example, the probability an employee will resign(target variable) given his attributes such as his age, salary, KPIs, etc.

Q What are the outputs of the logistic model and the logistic function?

Ans The logistic model outputs the logit, which refers to the log odds, while the logistic function transforms this output into probabilities. The logistic model calculates a weighted sum of the input features, and this result (the logit) is then passed through the logistic function to obtain a probability value between 0 and 1.

Department of Computer Engineering
Academic Term: July-November 2022

Class : B.*E. Computer* **Subject Name :*ML***
Semester : VII **Subject Code : *CSC701***

Practical No:	4
Title.	To Study and implement Decision Tree.
Date of Performance.	08-08-2025
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation

Indicator	**Very Poor**	**Poor**	**Average**	**Good**	**Excellent**
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('/content/PlayTennis.csv')
value=['Outlook','Temperature','Humidity','Wind']
df
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

```
mapping_outlook = {"Sunny": 1, "Overcast": 2, "Rain": 3}
mapping_temperature = {"Hot": 1, "Mild": 2, "Cool": 3}
mapping_humidity = {"High": 1, "Normal": 2}
mapping_wind = {"Strong": 1, "Weak": 2}
mapping_play = {"Yes": 1, "No": 2}

dataset = df.replace({
    "Outlook": mapping_outlook,
    "Temperature": mapping_temperature,
    "Humidity": mapping_humidity,
    "Wind": mapping_wind,
    "Play Tennis": mapping_play
})
```

```
/tmp/ipython-input-1521212356.py:7: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a
  dataset = df.replace({
```

```
len(df)
```

```
14
```

```
df.shape
```

```
(14, 5)
```

```
df.head()
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes

```
df.tail()
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

```
df.describe()
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
count	14	14	14	14	14
unique	3	3	2	2	2
top	Sunny	Mild	High	Weak	Yes
freq	5	6	7	8	9

```
#machine learning algorithms can only learn from numbers (int, float, doubles .. )
#so let us encode it to int
from sklearn import preprocessing
string_to_int= preprocessing.LabelEncoder()                    #encode your data
df=df.apply(string_to_int.fit_transform) #fit and transform it
df
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
0	2	1	0	1	0
1	2	1	0	0	0
2	0	1	0	1	1
3	1	2	0	1	1
4	1	0	1	1	1
5	1	0	1	0	0
6	0	0	1	0	1
7	2	2	0	1	0
8	2	0	1	1	1
9	1	2	1	1	1
10	2	2	1	0	1
11	0	2	0	0	1
12	0	1	1	1	1
13	1	2	0	0	0

```
# Define the feature columns
feature_cols = ['Outlook', 'Temperature', 'Humidity', 'Wind']

# Feature set (X) and label (y)
X = df[feature_cols]
y = df["Play Tennis"]  # Use correct column name with space
```

```
#To divide our data into training and test sets:
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
```

```
# perform training
from sklearn.tree import DecisionTreeClassifier                          # import the classifier
classifier =DecisionTreeClassifier(criterion="entropy", random_state=100)     # create a classifier object
classifier.fit(X_train, y_train)                                           # fit the classifier with X and Y data or
```

```
DecisionTreeClassifier(criterion='entropy', random_state=100)
```

```
#Predict the response for test dataset
y_pred= classifier.predict(X_test)
```

```
from sklearn.datasets import load_iris
import pandas as pd

# Load the iris dataset
iris = load_iris()

# Convert to a pandas DataFrame for easier manipulation
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df['target'] = iris.target
display(iris_df.head())
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
X_reg = iris_df[['sepal length (cm)']]  # Features need to be in a 2D array/DataFrame
y_reg = iris_df['sepal width (cm)']    # Target
```

```
# Split the data into training and testing sets
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

# Create a Linear Regression model
regressor = LinearRegression()
```

```
# Train the model
regressor.fit(X_train_reg, y_train_reg)

# Make predictions
y_pred_reg = regressor.predict(X_test_reg)
```

```
# Evaluate the model
print('Mean Squared Error:', mean_squared_error(y_test_reg, y_pred_reg))
print('R-squared:', r2_score(y_test_reg, y_pred_reg))
print('Coefficient:', regressor.coef_[0])
print('Intercept:', regressor.intercept_)
```

```
Mean Squared Error: 0.13961895650579023
R-squared: 0.024098626473972984
Coefficient: -0.058294182654555306
Intercept: 3.4003072894040876
```

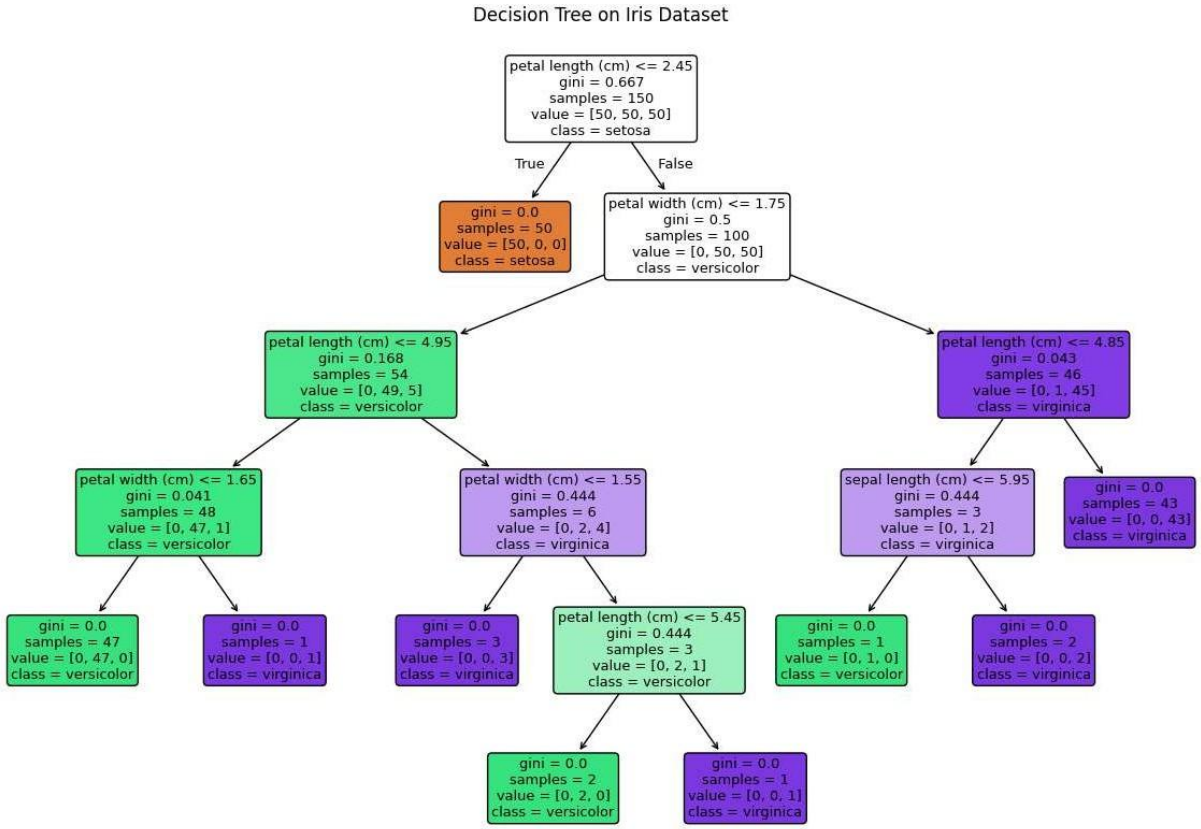
```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Define features (X) and target (y) for classification using the full Iris dataset
X_cls = iris_df[iris.feature_names]
y_cls = iris_df['target']
```

```
# Create a Decision Tree Classifier
dt_classifier = DecisionTreeClassifier(random_state=42)
# Train the classifier
dt_classifier.fit(X_cls, y_cls)
```

```
▾ DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

```
# Visualize the decision tree
plt.figure(figsize=(15, 10))
plot_tree(dt_classifier,
          feature_names=iris.feature_names,
```

```
            class_names=iris.target_names,
            filled=True,
            rounded=True)
plt.title("Decision Tree on Iris Dataset")
plt.show()
```



Department of Computer Engineering
Academic Term: July-November 2022

Class : B.*E. Computer* **Subject Name :*ML***
Semester : VII **Subject Code : *CSC701***

Practical No:	5
Title.	To Study and implement SVM.
Date of Performance.	22-08-2025
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation

Indicator	**Very Poor**	**Poor**	**Average**	**Good**	**Excellent**
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

In [ ]:
```
# Import pandas library
import pandas as pd
from matplotlib.colors import ListedColormap
import numpy as nm
import matplotlib.pyplot as mtp
from matplotlib.colors import ListedColormap
# initialize list of lists
data = [[3,1,1], [3,-1,1], [6,1,1], [6,-1,1], [1,0,0], [0,1,0], [0,-1,0], [-1,0,0]
data1 = [[2,2,1], [2,-2,1], [-2,-2,1], [-2,2,1], [1,1,0], [1,-1,0], [-1,-1,0], [-1
data2 = [[1,1,1], [2,2,1], [2,0,1],[0,0,0], [-1,0,0], [0,1,0]]

# Create the pandas DataFrame
df = pd.DataFrame(data, columns=['Col1', 'Col2', 'Output'])
df1 = pd.DataFrame(data1, columns=['Col1', 'Col2', 'Output'])
df2 = pd.DataFrame(data2, columns=['Col1', 'Col2', 'Output'])
# print dataframe.
df
```

Out[ ]:

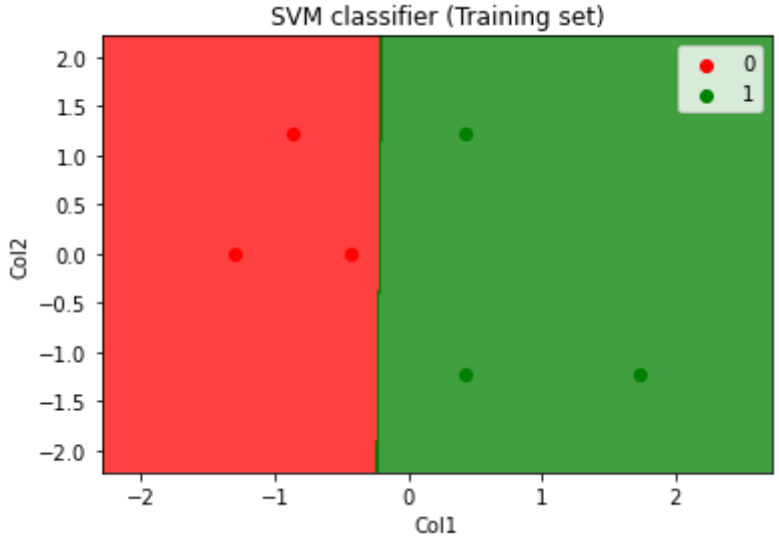
	Col1	Col2	Output
0	3	1	1
1	3	-1	1
2	6	1	1
3	6	-1	1
4	1	0	0
5	0	1	0
6	0	-1	0
7	-1	0	0

In [ ]:
```
def plot_svm(df) :
  #Extracting Independent and dependent Variable
  x= df.iloc[:, [0,1]].values
  y= df.iloc[:, 2].values
  # Splitting the dataset into training and test set.
  from sklearn.model_selection import train_test_split
  x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random
  #feature Scaling
  from sklearn.preprocessing import StandardScaler
  st_x= StandardScaler()
  x_train= st_x.fit_transform(x_train)
  x_test= st_x.transform(x_test)
  from sklearn.svm import SVC # "Support vector classifier"
  classifier = SVC(kernel='linear', random_state=0)
  classifier.fit(x_train, y_train)
      #Predicting the test set result
  y_pred= classifier.predict(x_test)
  #Creating the Confusion matrix
  from sklearn.metrics import confusion_matrix
  cm= confusion_matrix(y_test, y_pred)
  x_set, y_set = x_train, y_train
  x1, x2 = nm.meshgrid(nm.arange(start = x_set[:, 0].min() - 1, stop = x_set[:, 0]
  nm.arange(start = x_set[:, 1].min() - 1, stop = x_set[:, 1].max() + 1, step = 0.
  mtp.contourf(x1, x2, classifier.predict(nm.array([x1.ravel(), x2.ravel()]).T).re
  alpha = 0.75, cmap = ListedColormap(('red', 'green')))
  mtp.xlim(x1.min(), x1.max())
  mtp.ylim(x2.min(), x2.max())
```

```
  for i, j in enumerate(nm.unique(y_set)):
    mtp.scatter(x_set[y_set == j, 0], x_set[y_set == j, 1],
    c = ListedColormap(('red', 'green'))(i), label = j)
  mtp.title('SVM classifier (Training set)')
  mtp.xlabel('Col1')
  mtp.ylabel('Col2')
  mtp.legend()
  mtp.show()
        #Visulaizing the test set result

  x_set, y_set = x_test, y_test
  x1, x2 = nm.meshgrid(nm.arange(start = x_set[:, 0].min() - 1, stop = x_set[:, 0]
  nm.arange(start = x_set[:, 1].min() - 1, stop = x_set[:, 1].max() + 1, step = 0.
  mtp.contourf(x1, x2, classifier.predict(nm.array([x1.ravel(), x2.ravel()]).T).re
  alpha = 0.75, cmap = ListedColormap(('red','green' )))
  mtp.xlim(x1.min(), x1.max())
  mtp.ylim(x2.min(), x2.max())
  for i, j in enumerate(nm.unique(y_set)):
    mtp.scatter(x_set[y_set == j, 0], x_set[y_set == j, 1],
    c = ListedColormap(('red', 'green'))(i), label = j)
  mtp.title('SVM classifier (Test set)')
  mtp.xlabel('Col1')
  mtp.ylabel('Col2')
  mtp.legend()
  mtp.show()
```

In [ ]:
```
plot_svm(df)
```

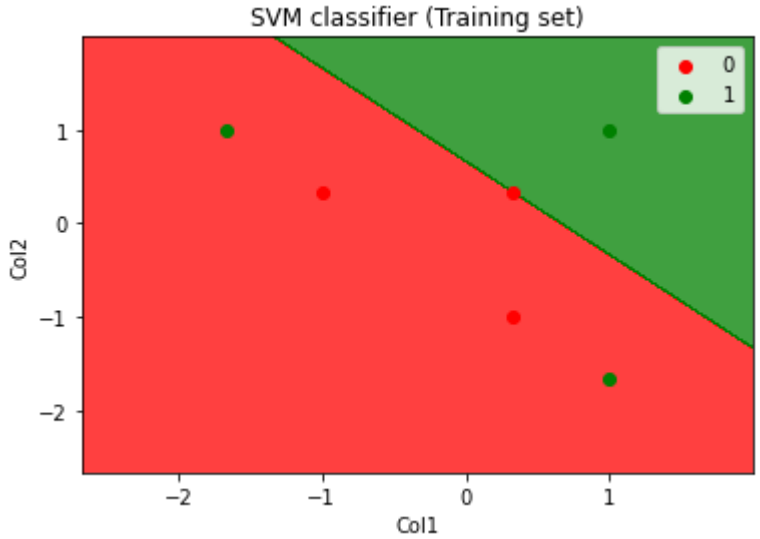
```
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA
sequence, which should be avoided as value-mapping will have precedence in case it
s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi
de a 2-D array with a single row if you intend to specify the same RGB or RGBA val
ue for all points.
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA
sequence, which should be avoided as value-mapping will have precedence in case it
s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi
de a 2-D array with a single row if you intend to specify the same RGB or RGBA val
ue for all points.
```



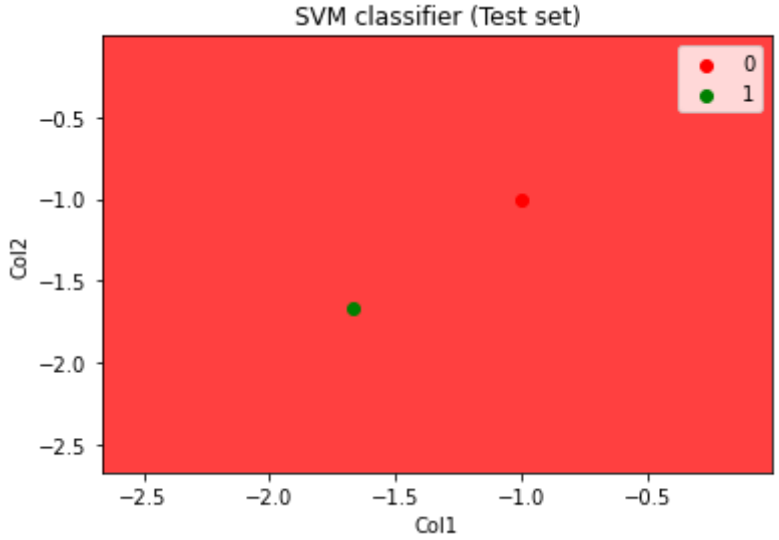
```
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
```

In [ ]:
```
plot_svm(df1)
```

```
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
```

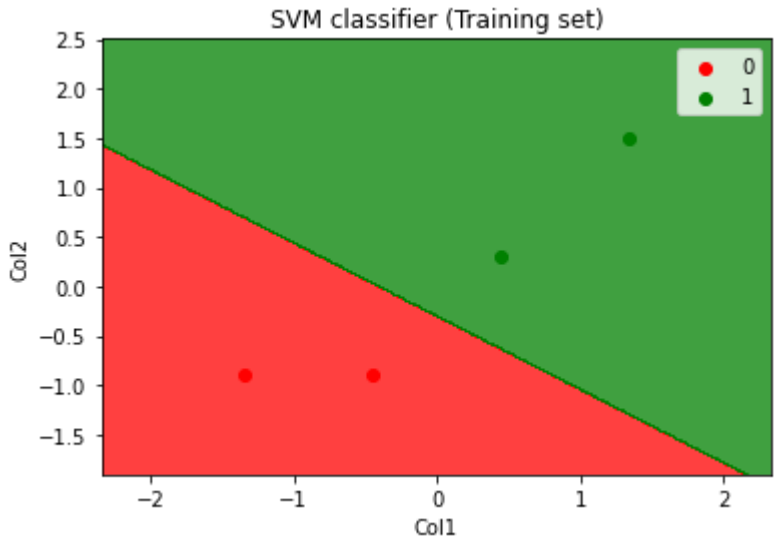


```
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
```

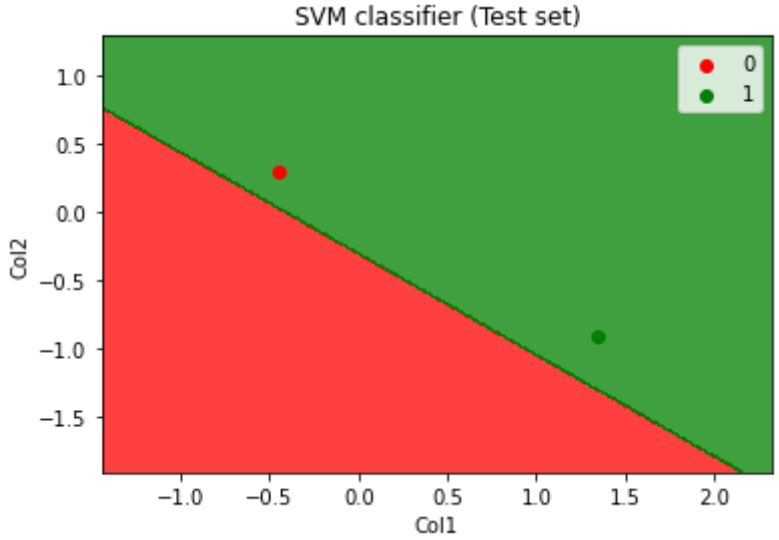


In [ ]:
```
plot_svm(df2)
```

```
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
```



```
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
WARNING:matplotlib.axes._axes:*c* argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value-mapping will have precedence in case it s length matches with *x* & *y*.  Please use the *color* keyword-argument or provi de a 2-D array with a single row if you intend to specify the same RGB or RGBA val ue for all points.
```



9913 ML - Postlab - 5

Q.1 What is geometric intention behind SVM?

→ The main intention is to find the optimal boundary - hyperplane, that is the best line which seperates data points of classes while maximizing margin between them.

Q.2 How would we explain convex hull in light of SVM

→ we simply build a convex hull for class A and B and draw a perpendicular on the shortest distance between closest points of both these hulls.

Q.3 what do know about hard margin SVM and short margin SVM?

→ If a point a classifies the equation $(wt * a + b) \geq 1$ then a is correctly classified. So if the points are linearly seperable then only the hyperplane is able to distinguish between them

Q.4 what is hinge loss?

→ Hinge loss is a loss function which penalizes the SVM model for inaccurate prediction.
if $y_i(wt * a + b) \geq 1$, hinge loss is '0' i.e. points are correctly classified. when $y_i(wt * a + b) < 1$
∴ hinge loss increases massively.
As $y_i(wt * a + b)$ increases with every misclassified point, the upper bound of hinge loss $\{1 - y_i(wt * a$ also increases exponentially.

Department of Computer Engineering
Academic Term: July-November 2022

Class : B.*E. Computer* **Subject Name :*ML***
Semester : VII **Subject Code : *CSC701***

Practical No:	6
Title.	To Study and implement clustering algorithm
Date of Performance.	12-09-2025
Roll No:	9913
Name of the Student:	Mark Lopes

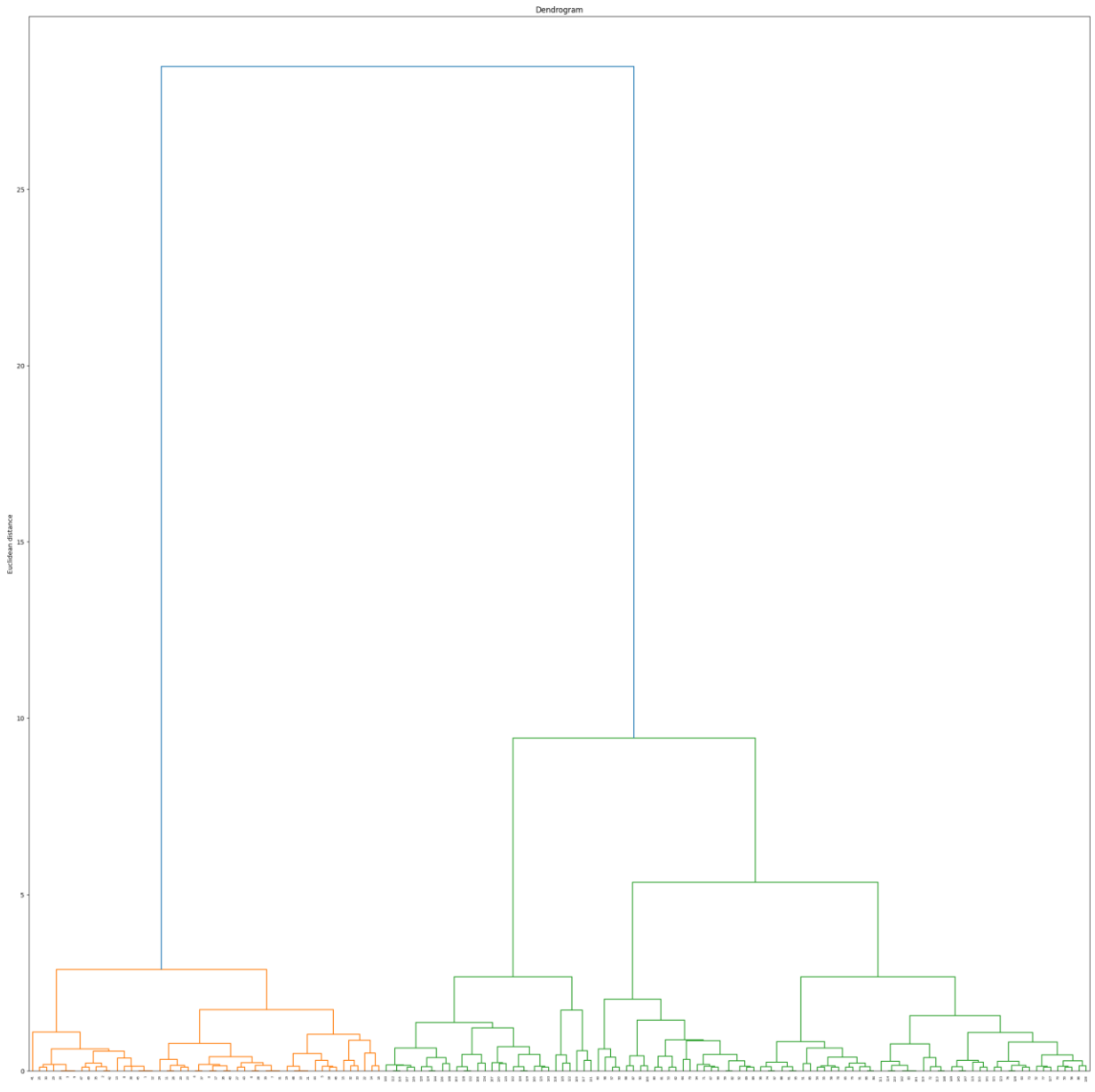
Evaluation

Indicator	**Very Poor**	**Poor**	**Average**	**Good**	**Excellent**
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

```
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram ,linkage
```

```
data = load_iris()
df = data.data
df = df[:,1:3]
Z = linkage(df, method = "ward")
plt.figure(figsize=(30,30))
dendro = dendrogram(Z)
plt.title("Dendrogram")
plt.ylabel("Euclidean distance")
```

```
Text(0, 0.5, 'Euclidean distance
```



2913 Postlab NL Exp6
Mark Lopes Batch-C

Q. What is Similarity based clustering
→ It is a technique in which data points are grouped into clusters based on how similar they are to each other. It uses measures such as Euclidean distance, cosine similarity or Jaccard ~~distance~~ index to determine closeness between objects. Points with high similarity are placed in same cluster, while dissimilar points fall into different cluster.

Q. Why do we need to perform significance testing in clustering.
→ It is important to ensure that the identified clusters represent genuine patterns in data rather than random groupings. Since clustering is unsupervised and lacks predefined labels, there is a risk of interpreting noise as structure. By performing significance testing, we can validate the quality and distinctiveness of clusters.

Q. Give example of confusion clustering to solve real-life problem.
→ It can be applied in healthcare for disease diagnosis. Patients symptoms and medical test results may overlap across different diseases creating confusion in classification. By using confusion clustering, patients of same patterns can be grouped together.

Q. difference between hard and soft clustering

→ • Hard clustering: Each data point has exactly one cluster. No overlaps, once a point is in a cluster, it cannot belong to another.
• Soft clustering: Each data point can belong to multiple clusters with a certain probability or degree of membership.

Q. What are challenges associated with clustering

→ There are multiple challenges:-
1. choosing no of clusters in advance
2. Selecting an appropriate similarity measures
3. Handling high-dimensional data
4. Dealing with noise and outliers
5. Scalability to large datasets

Q.

Department of Computer Engineering
Academic Term: July-November 2022

Class : B.*E. Computer* **Subject Name :*ML***
Semester : VII **Subject Code : *CSC701***

Practical No:	7
Title.	**To Study and implement Ensemble Techniques**
Date of Performance.	19/09/2025
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation

Indicator	**Very Poor**	**Poor**	**Average**	**Good**	**Excellent**
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

Total Marks

Signature of the Teacher

AY - 2022-2023

Fr. Conceicao Rodrigues College of Engineering

Bandstand Bandra (West) Mumbai 400053 **0**

Department of Computer Engineering

Experiment 7

To Study and implement Ensemble Techniques

II-THEORY

Introduction To Ensemble Learning

Ensemble Learning Techniques in Machine Learning, Machine learning models suffer bias and/or variance. Bias is the difference between the predicted value and actual value by the model. Bias is introduced when the model doesn't consider the variation of data and creates a simple model. The simple model doesn't follow the patterns of data, and hence the model gives errors in predicting training as well as testing data i.e. the model with high bias and high variance.

When the model follows even random quirks of data, as pattern of data, then the model might do very well on training dataset i.e. it gives low bias, but it fails on test data and gives high variance.

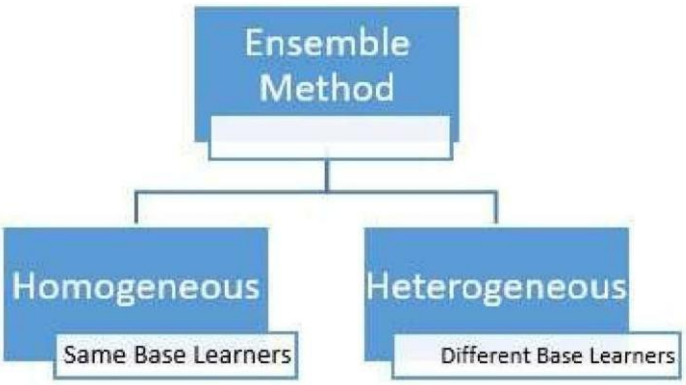
Therefore, to improve the accuracy (estimate) of the model, ensemble learning methods are developed. Ensemble is a machine learning concept, in which several models are trained using machine learning algorithms. It combines low performing classifiers (also called as weak learners or base learner) and combine individual model prediction for the final prediction.

On the basis of type of base learners, ensemble methods can be categorized as homogeneous and heterogeneous ensemble methods. If base learners are same, then it is a homogeneous ensemble method. If base learners are different then it is a heterogeneous ensemble method.

Fr. Conceicao Rodrigues College of Engineering
Bandstand Bandra (West) Mumbai 400 53

Department of Computer Engineering

AY - 2022-2023



Ensemble Learning Methods

Ensemble techniques are classified into three types:

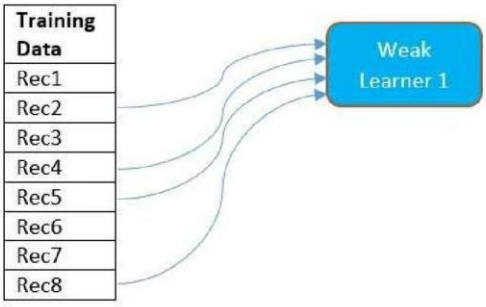
1. Bagging
2. Boosting
3. Stacking

Bagging

Consider a scenario where you are looking at the users' ratings for a product. Instead of approving one user's good/bad rating, we consider average rating given to the product. With average rating, we can be considerably sure of quality of the product. Bagging makes use of this principle. Instead of depending on one model, it runs the data through multiple models in parallel, and average them out as model's final output.

What is Bagging? How it works?

- Bagging is an acronym for **Bootstrapped Aggregation**. Bootstrapping means random selection of records with replacement from the training dataset. 'Random selection with replacement' can be



Fr. Conceicao Rodrigues College of Engineering

Bandstand Bandra (West) Mumbai 400053

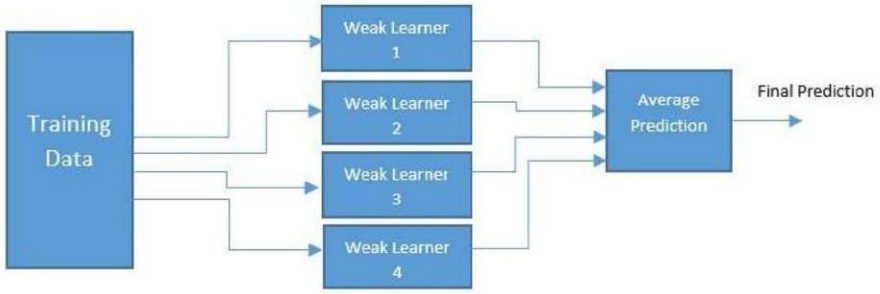
Department of Computer Engineering

AY - 2022-2023

explained as follows:

a. Consider that there are 8 samples in the training dataset. Out of these 8 samples, every weak learner gets 5 samples as training data for the model. These 5 samples need not be unique, or non-repetitive.
b. The model (weak learner) is allowed to get a sample multiple times. For example, as shown in the figure, Rec5 is selected 2 times by the model. Therefore, weak learner1 gets Rec2, Rec5, Rec8, Rec5, Rec4 as training data.
c. All the samples are available for selection to next weak learners. Thus all 8 samples will be available for next weak learner and any sample can be selected multiple times by next weak learners.

- Bagging is a **parallel method**, which means several weak learners learn the data pattern independently and simultaneously. This can be best shown in the below diagram:



1. The output of each weak learner is averaged to generate final output of the model.
2. Since the weak learner's outputs are averaged, this mechanism helps to reduce variance or variability in the predictions. However, it does not help to reduce bias of the model.
3. Since final prediction is an average of output of each weak learner, it means that each weak learner has equal say or weight in the final output.

To summarize:

1. Bagging is Bootstrapped Aggregation
2. It is Parallel method
3. Final output is calculated by averaging the outputs produced by individual weak learner
4. Each weak learner has equal say
5. Bagging reduces variance

Boosting

We saw that in bagging every model is given equal preference, but if one model predicts data more correctly than the other, then higher weightage should be given to this model over the other. Also, the model should attempt to reduce bias. These concepts are applied in the second ensemble method that we are going to learn, that is Boosting.

Fr. Conceicao Rodrigues College of Engineering

Bandstand Bandra (West) Mumbai 400053

Department of Computer Engineering

AY - 2022-2023

What is Boosting?

1. To start with, boosting assigns equal weights to all data points as all points are equally important in the beginning. For example, if a training dataset has N samples, it assigns weight = 1/N to each sample.
2. The weak learner classifies the data. The weak classifier classifies some samples correctly, while making mistake in classifying others.
3. After classification, sample weights are changed. Weight of correctly classified sample is reduced, and weight of incorrectly classified sample is increased. Then the next weak classifier is run.
4. This process continues until model as a whole gives strong predictions.

Note: Adaboost is the ensemble learning method used in binary classification only.

How Adaboost Algorithm Works?

Consider below training data for heart disease classification.

Blocked Arteries	Chest Pain	Weight	Heart Disease
Y	Y	200	Y
Y	N	185	Y
N	Y	200	Y
Y	Y	160	Y
Y	N	140	N
Y	N	130	N
N	Y	170	N
Y	Y	170	N

1. Initialize Weights To All Training Points

First step is to assign equal weights to all samples as all samples are equally important. Always, sum of weights of all samples equals 1. There are 8 samples, so each sample will get weight = 1/8 = 0.125

Since all samples are equally important to start with, all samples get equal weight: 1 / total number of samples = 1/8

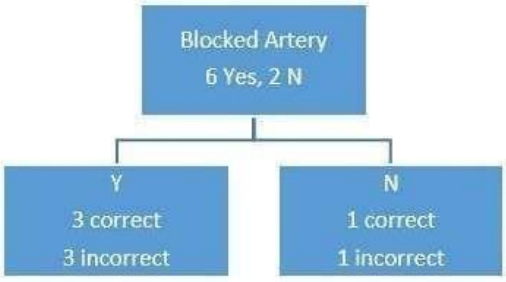
Blocked Arteries	Chest Pain	Weight	Heart Disease	Weights Assigned
Y	Y	200	Y	1/8
Y	N	185	Y	1/8
N	Y	200	Y	1/8
Y	Y	160	Y	1/8
Y	N	140	N	1/8
Y	N	130	N	1/8
N	Y	170	N	1/8
Y	Y	170	N	1/8

2. Create Stump

After assigning weight, next step is to create stumps. Stump is a decision tree with one node and two leaves. Adaboost creates forest of decision stumps. To create stump, only one attribute should be chosen. But, it is not randomly selected. The attribute that does the best job of classifying the sample is selected first.

So let's see how each attribute classifies the samples.

Blocked Artery:

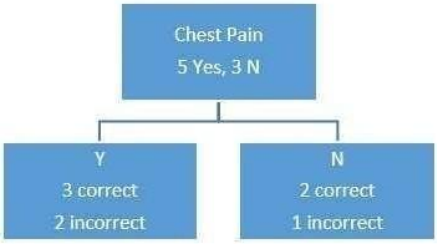


Chest Pain

Fr. Conceicao Rodrigues College of Engineering

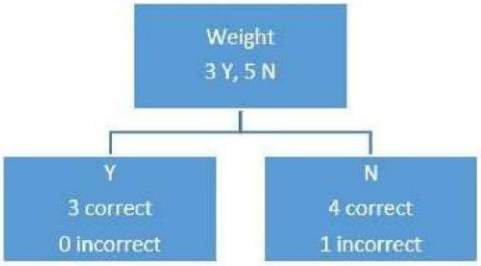
Bandstand Bandra (West) Mumbai 400053

Department of Computer Engineering



Weight:

Weight is continuous quantity, and there is a method to find threshold for continuous quantity in decision tree algorithm. According to that method, the threshold is 175. Based on this condition draw the decision stump.



From above classification, it can be seen that

- 'Blocked Arteries' as a stump made 4 errors.
- 'Chest Pain' as a stump made 3 errors.
- 'Weight' as a stump made 1 error.

Since, the attribute 'Weight' made least number of errors, 'Weight' is the best stump to start with.

Note: Since this was attribute selection for first stump, we need not consider weight of each sample. This is because, to start with, all samples have equal weights.

To select next stump, both, weight of samples and errors made by that attribute while classifying the samples, will be considered.

3. Calculate Total Error And Voting Power (Amount Of Say) Of The Stump

After choosing best attribute for stump, next step is to calculate the voting power (amount of say) of that stump in the final classification. Formula to calculate amount of say is:

Amount of say = ½ * *ln*((1-Total Error) / Total Error)

Where *ln* is Natural Log
Total Error = Number of misclassified samples * Weight of sample.
Since the stump misclassified only 1 sample, Total Error made by stump = weight of 1 sample = 1/8 = 0.125
Substituting Total Error in the equation,
Amount of say = ½ * *ln*((1-0.125)/0.125)
= 0.9729

4. Start To Build Final Classifier Function

The Adaboost classifier then starts building a function that will assign this 'amount of say' as voting power of the stump. It will assign 0.9729 as voting power to the stump created for 'Weight' attribute.

The function h(x) = 0.9729* 'Weight' as stump

5. Update New Weights For The Classifier

After calculation of amount of say, next step is to create another stump. However, before that adjust the weights assigned to each sample. The samples that were correctly classified in previous stump should get less importance as their classification is over, and the samples that were not correctly classified should get more weight in the next step.

The formula to assign new weight for **correctly classified** samples is:

$$W_{new} = W_{old} / 2(1\text{-Total Weight})$$

The formula to assign new weight for **incorrectly classified** samples is:

$$W_{new} = W_{old} / (2*\text{Total Weight})$$

New weights of correctly classified samples =
$W_{new} = (0.128) / 2(1-0.125) = 0.07$
New weights of incorrectly classified samples =
$W_{new} = (0.125) / (2*0.125) = 0.5$
Substitute new weights:

Blocked Arteries	Chest Pain	Weight	Heart Disease	Weights Assigned
Y	Y	200	Y	0.07
Y	N	185	Y	0.07
N	Y	200	Y	0.07
Y	Y	160	Y	0.5
Y	N	140	N	0.07
Y	N	130	N	0.07
N	Y	170	N	0.07
Y	Y	170	N	0.07

From above classification, it can be seen that

- 'Blocked Arteries' as a stump made 4 errors. Therefore, Total Error for 'Blocked Arteries' = 0.07*4 = 0.28

- 'Chest Pain' as a stump made 3 errors. Therefore, Total Error for 'Chest Pain' = 0.07*3 = 0.21
- 'Weight' as a stump made 1 error. Therefore, Total error for weight = 1*0.5 = 0.5.

Note: We have already considered 'Weight' attribute for first stump creation. So this attribute need not be considered again. However, it is considered here to show effect of 'Assigned Weight' of the sample on Total Error calculation.

6. Calculate Voting Power (Amount of Say) and Add to Final Classifier Function

Since Chest Pain classifier has least Total Error, this attribute is selected as second stump.

The Amount of Say = ½ ln(1-0.21)/0.21 =0.66

Add this classifier with its weight to the function.

The function h(x) = 0.9729* 'Weight' as stump + 0.66*'Chest Pain' as stump

7. Assign New Weights and Repeat

Again, assign new weights to the correctly classified samples with formula:

$W_{new} = W_{old} / 2(1\text{-total weight})$
And incorrectly classified samples with the formula:
$W_{new} = W_{old} / (2*\text{total weight})$

We repeat this process until

1. Enough rounds of this process are done to create strong classifier
2. No good classifiers left for making predictions.

When it comes to classifying data point, even if one classifier misclassifies a point, the voting powers of other classifiers override it making strong predictions.

There is another method to train Adaboost algorithm. In this method, new sample dataset is created in every iteration. The sample dataset is of same size as original dataset, and it contains some samples that are repeatedly selected. Samples with higher weights (i.e. samples that are incorrectly classified in previous round), will be selected repeatedly. The weight vector is normalized.

To summarize:

1. Adaboost is a sequential method where one weak learner runs after other.
2. Weak learners are stumps i.e. decision tree with 2 nodes.
3. Voting power is assigned to stumps based on how correctly it classifies the data.
4. Boosting reduces bias

Other Boosting Algorithms

Apart from Adaboost, there is other boosting algorithm – Gradient Boosting or Gradient Boosting Machine (GBM). The main idea behind this algorithm is to create decision trees with smaller leaves and scale the tree with a learning rate.

Decision tree algorithm is prone to overfitting and leads to high variance. Hence prediction made by the tree is scaled (or multipled) with the learning rate. This reduces training accuracy, but provides better results in long run. Gradient Boosting is used in classification as well as regression problems.

The open source implementations Gradient Boosting algorithms are XGBoost, LightGBM that are more regularized, and efficient.

III IMPLEMENT THE FOLLOWING PROBLEM STATEMENTS

IV CODE WITH OUTPUT

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
df = pd.read_csv("dataset.csv")  # ensure dataset.csv is uploaded
df.head()
```

	Unnamed: 0	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	ener
0	0	5SuOikwiRyPMVoIQDJUgSV	Gen Hoshino	Comedy	Comedy	73	230666	False	0.676	0.46
1	1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55	149610	False	0.420	0.16
2	2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN	To Begin Again	To Begin Again	57	210826	False	0.438	0.35
3	3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Sou...	Can't Help Falling In Love	71	201933	False	0.266	0.05
4	4	5vjLSffimiIP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	82	198853	False	0.618	0.44

5 rows × 21 columns

```
df = df.drop(columns=["Unnamed: 0", "track_id", "artists", "album_name", "track_name"])

# Encode target variable (track_genre)
label_encoder = LabelEncoder()
df["track_genre"] = label_encoder.fit_transform(df["track_genre"])

# Features (X) and Target (y)
X = df.drop(columns=["track_genre"])
y = df["track_genre"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training set shape:", X_train.shape)
print("Test set shape:", X_test.shape)
```

```
Training set shape: (91200, 15)
Test set shape: (22800, 15)
```

```
# ========================
# 📌 Cell 4: Train Bagging Classifier (Fixed)
# ========================
bagging_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),  # updated param
    n_estimators=50,       # number of trees
    random_state=42,
    n_jobs=-1
)

# Train the model
bagging_model.fit(X_train, y_train)

print("✅ Model training complete!")
```

```
✅ Model training complete!
```

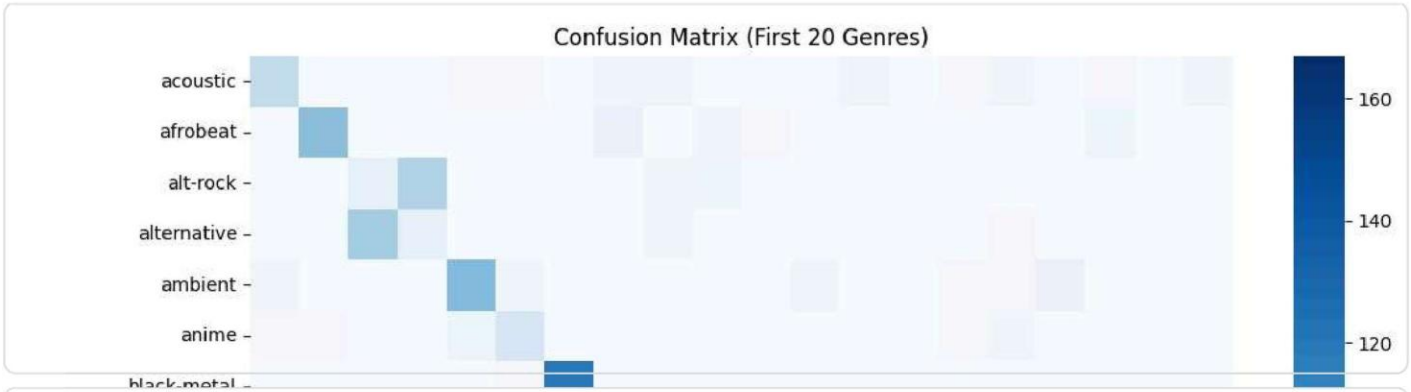
```
# ========================
# 📌 Cell 5: Evaluate Model (Fixed)
# ========================
# Predictions
y_pred = bagging_model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Classification Report (for ALL classes)
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=label_encoder.classes_))
```

```
              trance       0.40      0.40      0.40       200
           trip-hop        0.29      0.26      0.28       200
            turkish        0.25      0.33      0.29       200
        world-music        0.37      0.34      0.35       200

           accuracy                             0.30     22800
          macro avg        0.30      0.30      0.30     22800
       weighted avg        0.30      0.30      0.30     22800
```

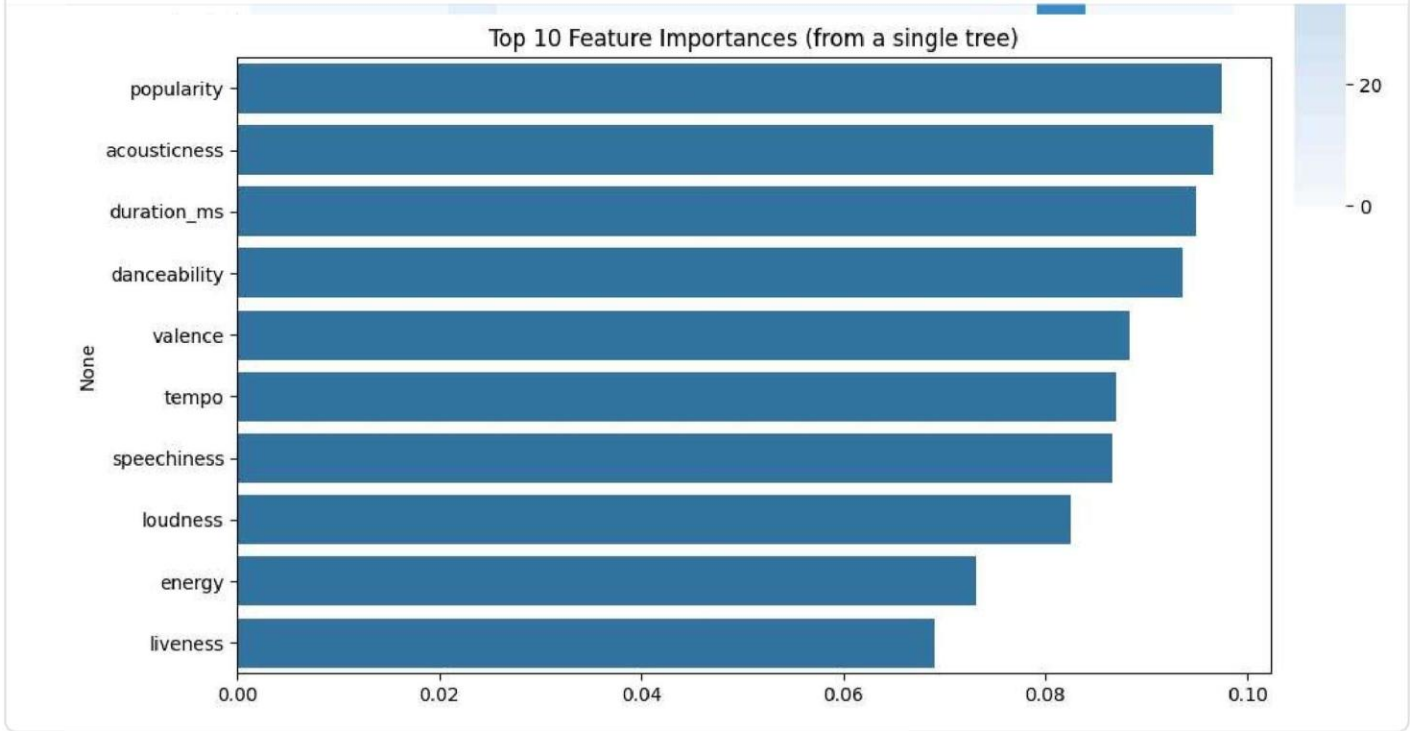
```
# ========================
# 📌 Cell 6: Predict on New Data (Optional)
# ========================
# Example: Take first 5 test samples and predict their genres
sample = X_test.iloc[:5]
predictions = bagging_model.predict(sample)

print("Predicted Genres:", label_encoder.inverse_transform(predictions))
```

```
Predicted Genres: ['disco' 'gospel' 'dancehall' 'j-rock' 'techno']
```

```
# ========================
# 📌 Cell 7: Confusion Matrix Heatmap
# ========================
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Compute confusion matrix (for all classes)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(12, 10))
sns.heatmap(cm[:20, :20], cmap="Blues", annot=False, fmt="d",
            xticklabels=label_encoder.classes_[:20],
            yticklabels=label_encoder.classes_[:20])
plt.title("Confusion Matrix (First 20 Genres)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
# ========================
# 📌 Cell 8: Feature Importance
# ========================
import numpy as np

# Since BaggingClassifier doesn't have feature_importances_ directly,
# we can take it from one of its Decision Trees
tree = bagging_model.estimators_[0]

importances = tree.feature_importances_
indices = np.argsort(importances)[::-1]
features = X.columns

plt.figure(figsize=(10,6))
sns.barplot(x=importances[indices][:10], y=features[indices][:10])
plt.title("Top 10 Feature Importances (from a single tree)")
plt.show()
```



Start coding or generate with AI.

DATE

PAGE NO.

9913ML- postlab -7

Q.1 what are weak learners?

→ Weak learners are simple models in ML that perform just better than random chance. They are usually high bias, low complexity, models meaning they don't capture all patterns in data and individually make many errors.

Q-2 How is random forest related to decision tree?

→ Random forest is directly related to decision
i) decision tree : Single tree that makes predictions by splitting data based on Feedback

ii) Random forest : An ensemble of decision trees Each tree is trained on random subset of data and a random subset of features
A random forest is like a team of decision tree working together.

Q.3 What is difference between begging and boosting?

→

Feature	Bagging	Boosting
i) Goal	Reduce variance	Reduce bias, variance
ii) Method	Train model in parallel	Train model in sequential
iii) Example	Random forest	Adaboost, GB
iv) Voting	Majority vote	weight vote.

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering

Academic Year 2025-2026

Rubrics for Lab Exp eriments

Class: B.E. Computer Engineering **Subject Name:** ML

Semster: VIII **Subject Code:** CSC701

Practical No.	8
Title:	To Study and implement Dimension Reduction
Date of Performance:	17-10-2025
Roll No.	9913
Student	Mark Lopes

Evaluation

Performance Indicator	Very Poor	Poor	Average	Good	Excellent
Timeline (2)	More than three sessions late(0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or on time(2)
Efforts (4)	N/A	N/A	Not completed(1)	Partially completed(1)	Completed(4)
Legibility (2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially understood the concept(1)	Understood the concept(2)
Total					

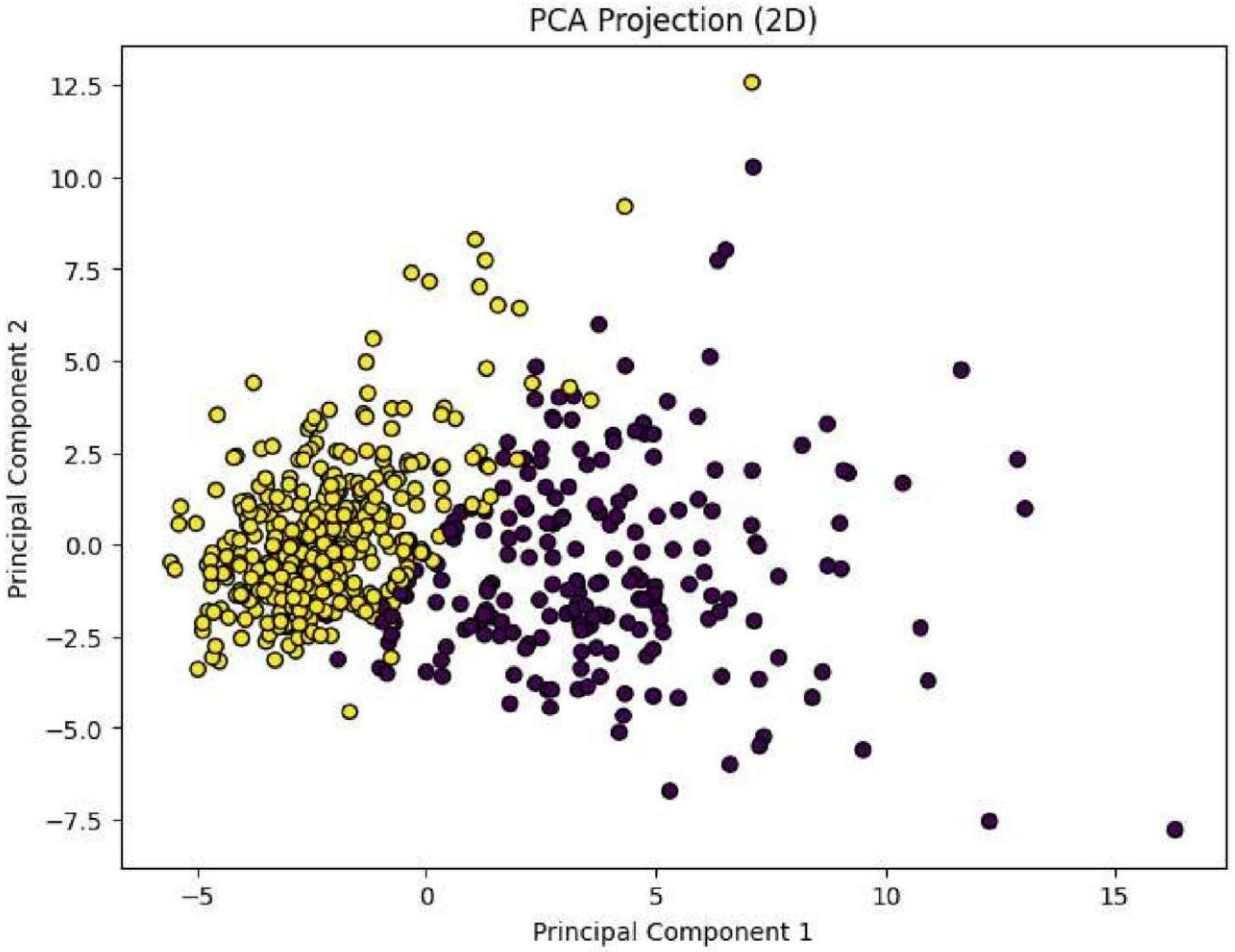
Signature of the Teacher:

```
# Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.feature_selection import VarianceThreshold
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

# ---------------------------------------------
# Load Dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print("Original Dataset Shape:", X.shape)

# ---------------------------------------------
# 1. Principal Component Analysis (PCA)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', edgecolor='k')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Projection (2D)')
plt.show()

# ---------------------------------------------
# 2. Low Variance Filter
selector = VarianceThreshold(threshold=0.01)
X_low_var = selector.fit_transform(X)
print("Shape after Low Variance Filter:", X_low_var.shape)

# ---------------------------------------------
# 3. High Correlation Filter
def correlation_filter(data, threshold=0.9):
    corr_matrix = data.corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bo
    to_drop = [column for column in upper.columns if any(upper[column] > thresho
```

```
        return data.drop(to_drop, axis=1)

    X_uncorr = correlation_filter(X)
    print("Shape after High Correlation Filter:", X_uncorr.shape)

    # -----------------------------------------
    # 4. Feature Selection using Random Forest
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X, y)
    feature_importances = pd.Series(model.feature_importances_, index=X.columns)
    top_features = feature_importances.nlargest(10).index
    X_rf = X[top_features]

    print("Top 10 Features from Random Forest:\n", top_features)

    # -----------------------------------------
    # 5. Autoencoder for Dimensionality Reduction
    # Normalize input
    X_norm = StandardScaler().fit_transform(X)

    # Define Autoencoder
    input_dim = X.shape[1]
    encoding_dim = 10  # reduced dimensions

    input_layer = Input(shape=(input_dim,))
    encoded = Dense(encoding_dim, activation='relu')(input_layer)
    decoded = Dense(input_dim, activation='sigmoid')(encoded)

    autoencoder = Model(input_layer, decoded)
    encoder = Model(input_layer, encoded)

    autoencoder.compile(optimizer='adam', loss='mse')

    # Train autoencoder
    autoencoder.fit(X_norm, X_norm, epochs=50, batch_size=32, shuffle=True, verbose=
```

```
    # Encode data
    X_encoded = encoder.predict(X_norm)
    print("Shape after Autoencoder Encoding:", X_encoded.shape)

    # Visualize encoded 2D space if encoding_dim = 2
    if encoding_dim == 2:
        plt.scatter(X_encoded[:, 0], X_encoded[:, 1], c=y, cmap='coolwarm', edgecolo
        plt.title("2D Encoded Space via Autoencoder")
        plt.xlabel("Encoded Feature 1")
        plt.ylabel("Encoded Feature 2")
        plt.show()
```

```
Original Dataset Shape: (569, 30)
```




```
Shape after Low Variance Filter: (569, 14)
Shape after High Correlation Filter: (569, 20)
Top 10 Features from Random Forest:
 Index(['worst area', 'worst concave points', 'mean concave points',
       'worst radius', 'worst perimeter', 'mean perimeter', 'mean concavity',
       'mean area', 'worst concavity', 'mean radius'],
      dtype='object')
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
Shape after Autoencoder Encoding: (569, 10)
```

Univariate EDA

```
# Step 1: Import required libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA

# Step 2: Load dataset
from sklearn.datasets import load_iris
```

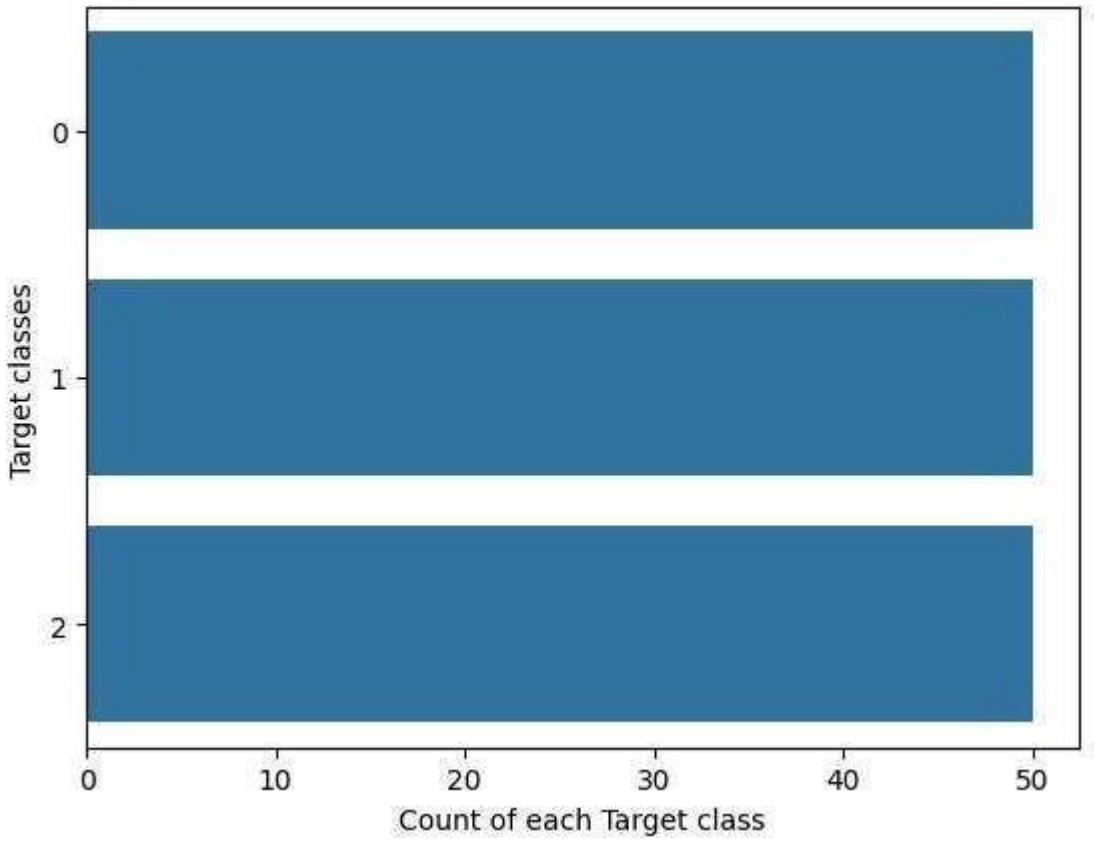
```
iris_data = load_iris()
X = pd.DataFrame(data=iris_data.data, columns=iris_data.feature_names)
y = pd.Series(iris_data.target, name='target')


# Step 3: Basic Info
print("Dataset Info:")
print(X.info())

# Step 4: Summary Statistics
print("\nDescriptive Statistics:")
print(X.describe())

# Step 5: First 5 rows
print("\nSample Data:")
print(X.head())
```

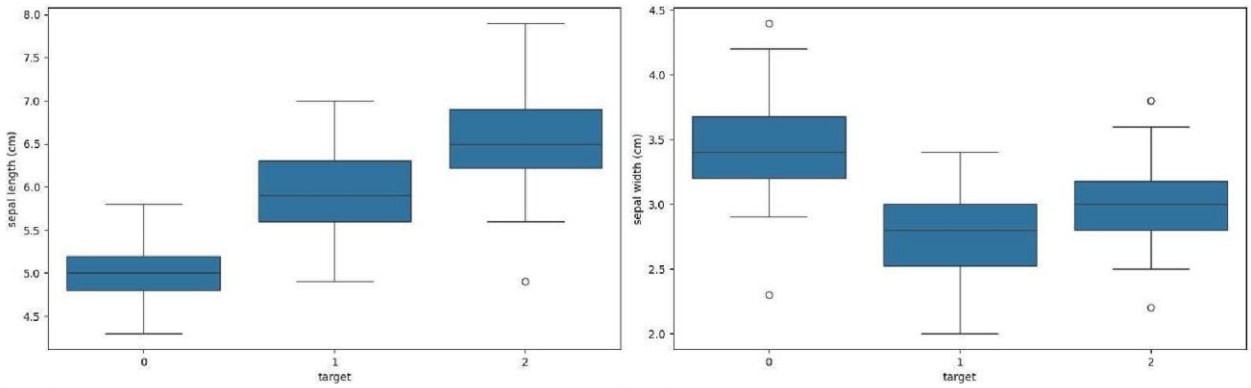
```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
dtypes: float64(4)
memory usage: 4.8 KB
None

Descriptive Statistics:
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000   
mean            5.843333          3.057333           3.758000   
std             0.828066          0.435866           1.765298   
min             4.300000          2.000000           1.000000   
25%             5.100000          2.800000           1.600000   
50%             5.800000          3.000000           4.350000   
75%             6.400000          3.300000           5.100000   
max             7.900000          4.400000           6.900000   

       petal width (cm)  
count        150.000000  
mean           1.199333  
std            0.762238  
min            0.100000  
25%            0.300000  
50%            1.300000  
75%            1.800000  
max            2.500000  

Sample Data:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
```

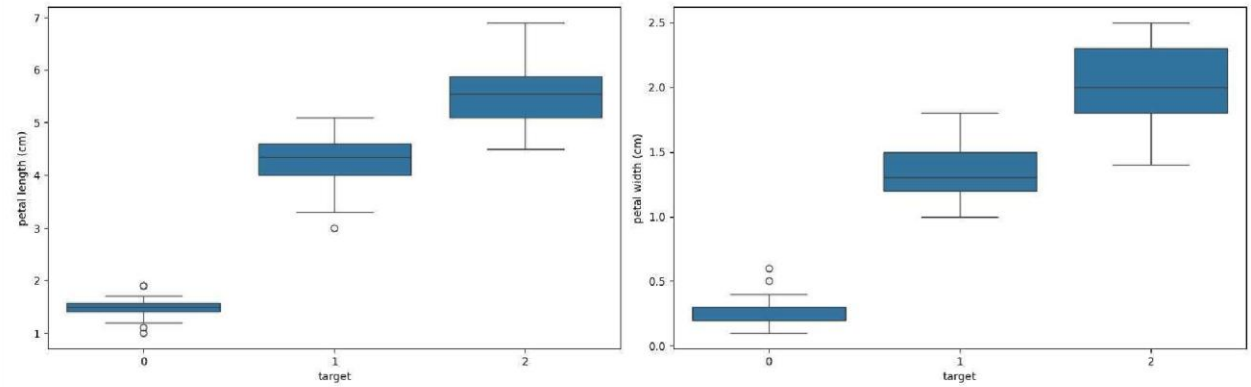
```
4                5.0               3.6               1.4               0.2
```

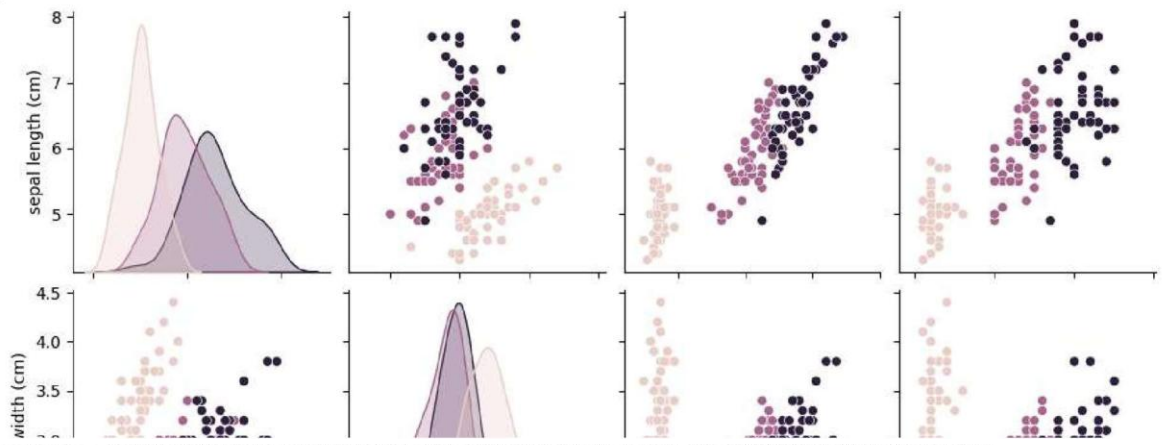
Bivariate EDA and Encoding

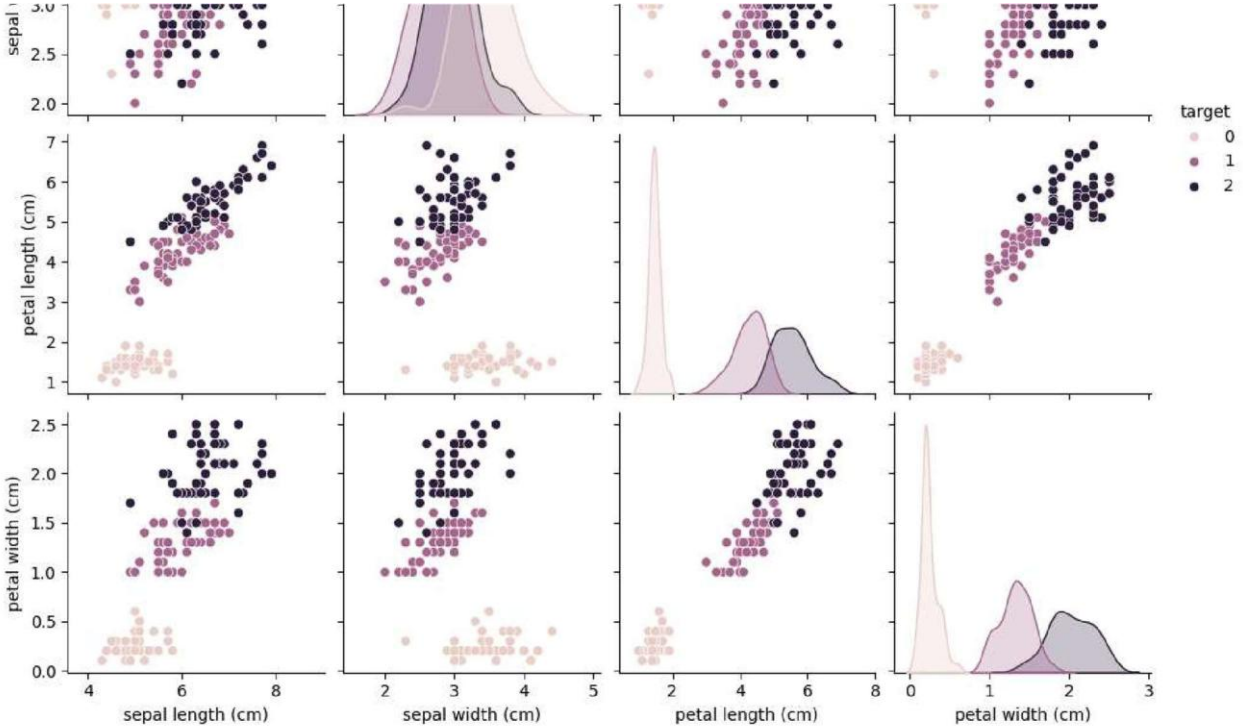
```
# Encode target labels (This step is not needed when using the target from sklea
# iris_data['species'] = iris_data['species'].replace({'Iris-setosa': 0, 'Iris-v

# Count plot
sns.countplot(y=y) # Use the 'y' variable for the target
plt.xlabel("Count of each Target class")
plt.ylabel("Target classes")
plt.show()

# Box plots by species
fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(16, 10))
features = X.columns # Use the column names from the features DataFrame 'X'

for i, feature in enumerate(features):
    row, col = i // 2, i % 2
    sns.boxplot(x=y, y=X[feature], ax=ax[row][col]) # Use 'y' for the x-axis and
plt.tight_layout()
plt.show()

# Pairplot
# sns.pairplot(iris_data, hue='species') # This will not work directly with the
# Let's create a DataFrame with both features and target for the pairplot
iris_df = pd.concat([X, y], axis=1)
sns.pairplot(iris_df, hue='target')
plt.show()


# Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(iris_df.corr(), annot=True, cmap='RdYlGn') # Use the combined datafr
plt.title("Feature Correlation Matrix")
plt.show()

# Histograms
X.hist(figsize=(12, 10), bins=15) # Use the features DataFrame 'X'
plt.suptitle("Feature Distributions")
plt.show()
```

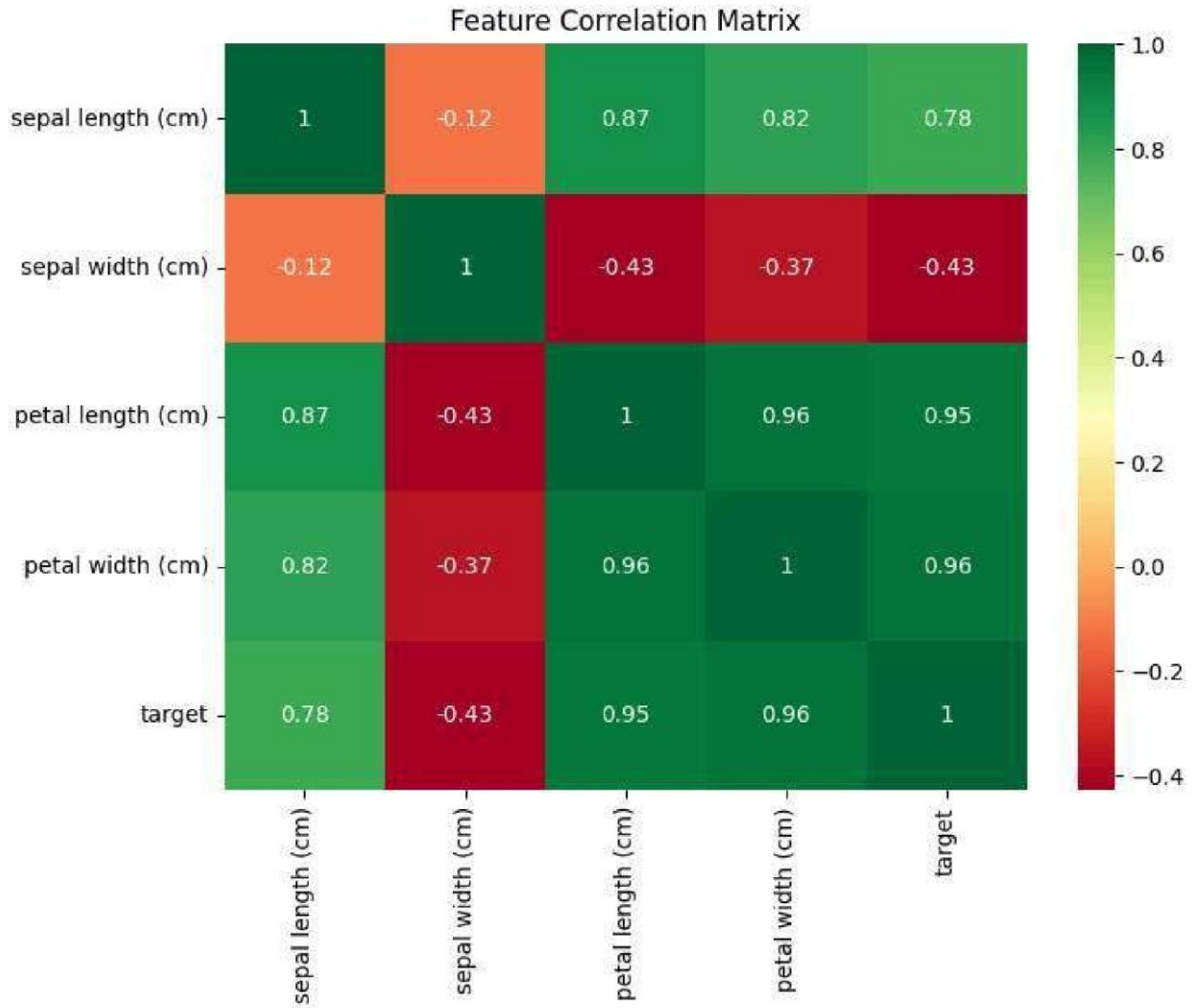

Target classes
0
1
2
0
10
20
30
40
50
Count of each Target class


sepal length (cm)
target
sepal width (cm)
target

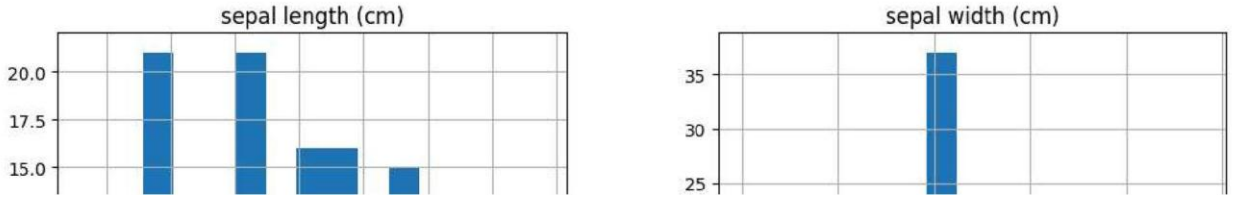

petal length (cm)
target
petal width (cm)
target

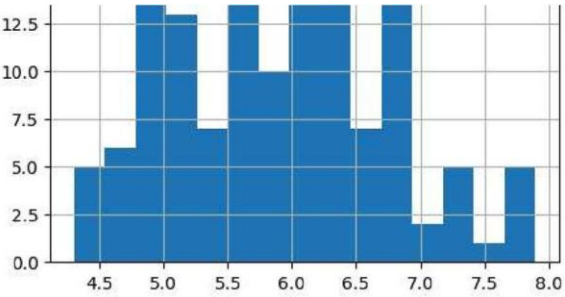

sepal length (cm)
width (cm)

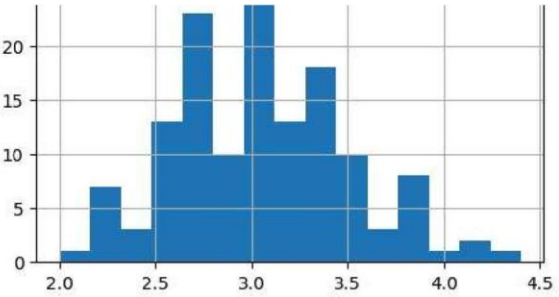


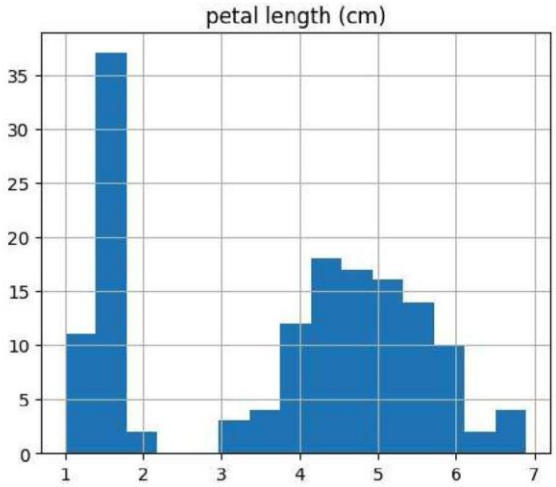


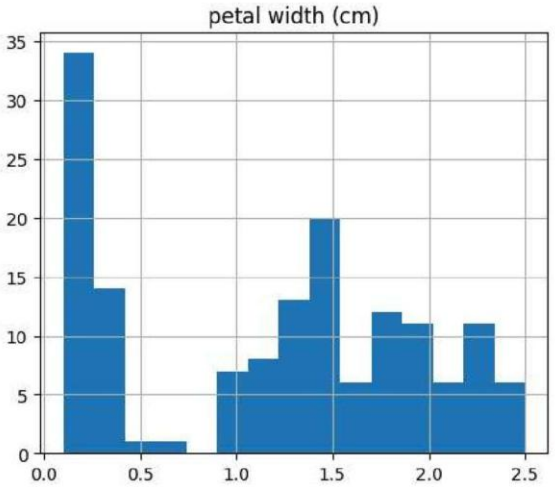
Feature Distributions




12.5
10.0
7.5
5.0
2.5
0.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0


20
15
10
5
0
2.0
2.5
3.0
3.5
4.0
4.5


petal length (cm)
35
30
25
20
15
10
5
0
1
2
3
4
5
6
7


petal width (cm)
35
30
25
20
15
10
5
0
0.0
0.5
1.0
1.5
2.0
2.5

🔹 Modelling 1️⃣ Without PCA

```
# Separate features and target - using X and y already created
# X = iris_data.drop('species', axis=1)
# y = iris_data['species']

# Normalize
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,

# Train KNN without PCA
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)

# Accuracy scores
print("Train score before PCA:", knn.score(X_train, y_train))
print("Test score before PCA:", knn.score(X_test, y_test))
```

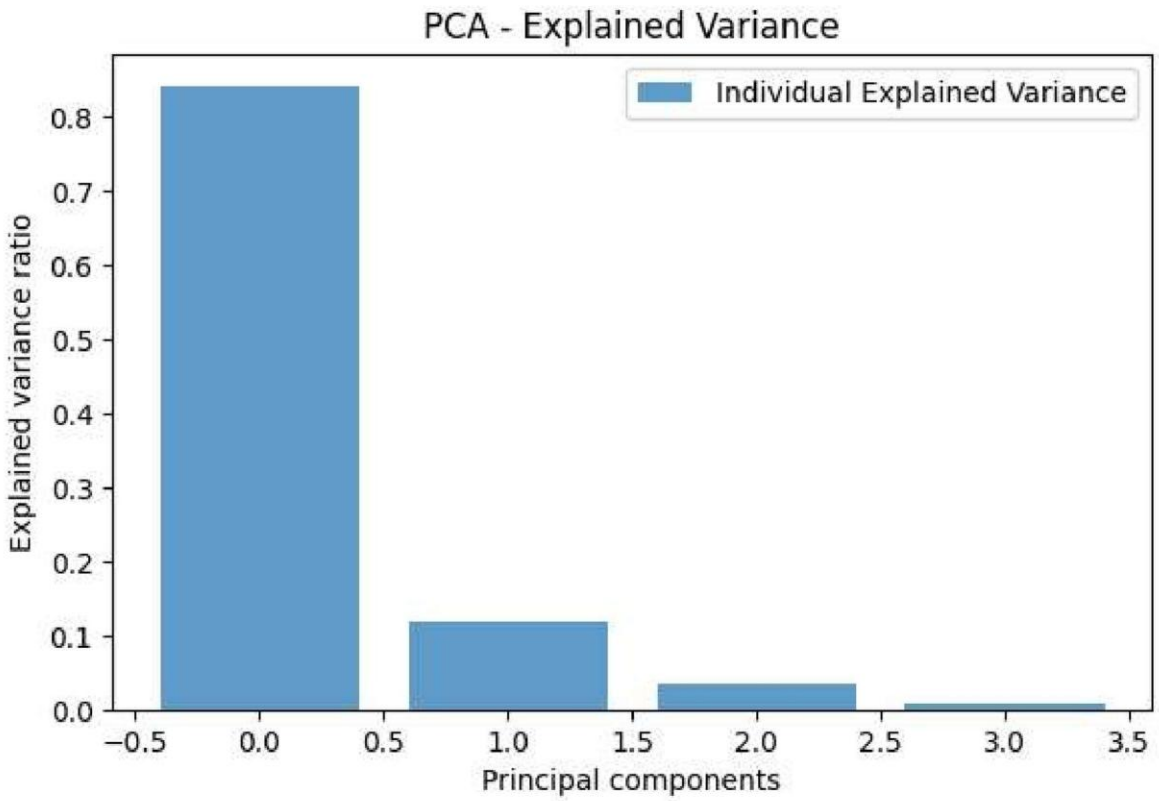
```
Train score before PCA: 0.9428571428571428
Test score before PCA: 1.0
```

2️⃣ With PCA

```
# Apply PCA
pca = PCA()
X_pca_full = pca.fit_transform(X_scaled)

# Explained Variance
explained_variance = pca.explained_variance_ratio_
print("Explained Variance Ratio:", explained_variance)

# Plot variance
plt.figure(figsize=(6, 4))
plt.bar(range(4), explained_variance, alpha=0.7, align='center', label='Individu
plt.ylabel('Explained variance ratio')
plt.xlabel('Principal components')
plt.legend(loc='best')
plt.tight_layout()
plt.title("PCA - Explained Variance")
plt.show()

# PCA with 3 components
pca = PCA(n_components=3)
X_reduced = pca.fit_transform(X_scaled)

# Train-Test split
X_train_pca, X_test_pca, y_train, y_test = train_test_split(X_reduced, y, test_s

# Train KNN on PCA data
```

```
knn_pca = KNeighborsClassifier(n_neighbors=7)
knn_pca.fit(X_train_pca, y_train)

print("Train score after PCA:", knn_pca.score(X_train_pca, y_train))
print("Test score after PCA:", knn_pca.score(X_test_pca, y_test))
```

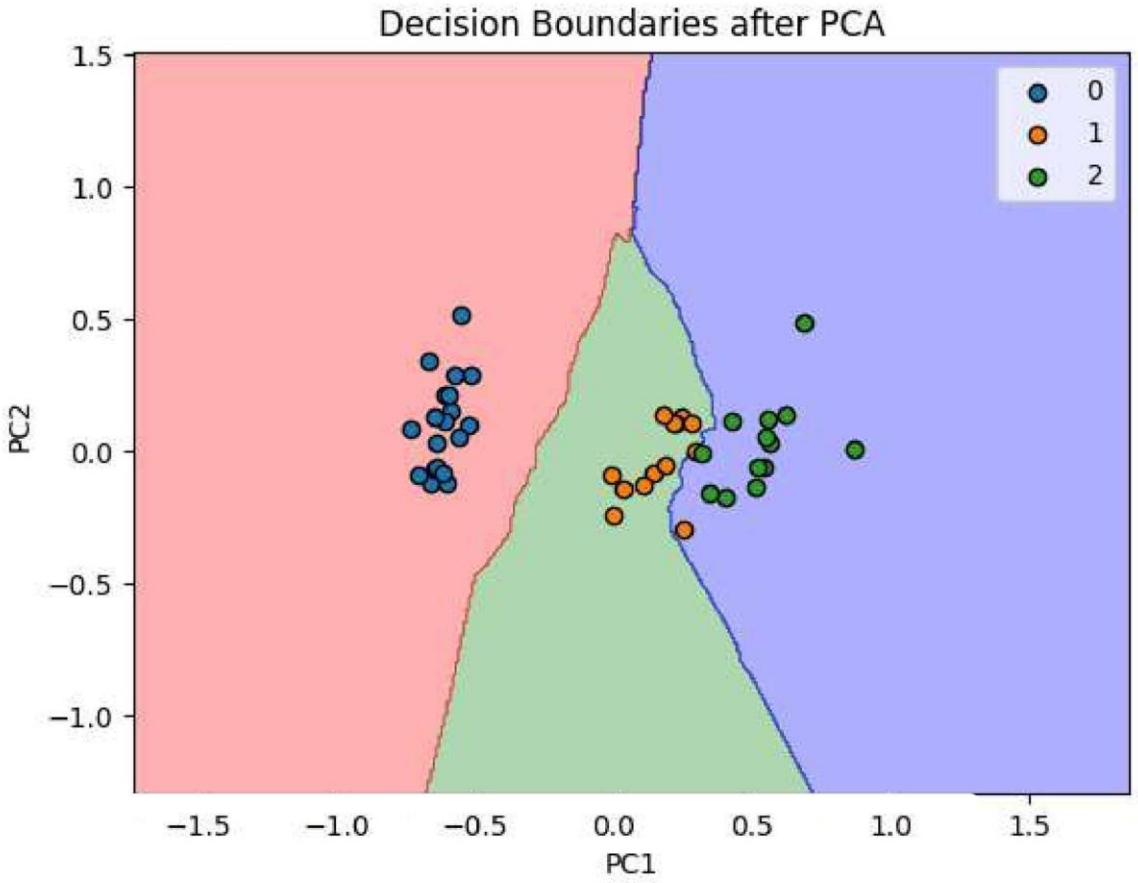
```
Explained Variance Ratio: [0.84136038 0.11751808 0.03473561 0.00638592]
```




```
Train score after PCA: 0.9523809523809523
Test score after PCA: 1.0
```

```
from matplotlib.colors import ListedColormap

# 2D plot if you use only 2 PCA components
X_2D = PCA(n_components=2).fit_transform(X_scaled)
X_train2D, X_test2D, y_train2D, y_test2D = train_test_split(X_2D, y, test_size=0
knn2D = KNeighborsClassifier(n_neighbors=7)
knn2D.fit(X_train2D, y_train2D)

# Meshgrid for plotting
X1, X2 = np.meshgrid(
    np.arange(X_test2D[:, 0].min() - 1, X_test2D[:, 0].max() + 1, 0.01),
    np.arange(X_test2D[:, 1].min() - 1, X_test2D[:, 1].max() + 1, 0.01)
)
plt.contourf(X1, X2, knn2D.predict(np.c_[X1.ravel(), X2.ravel()]).reshape(X1.sha
             alpha=0.3, cmap=ListedColormap(('red', 'green', 'blue')))

for i, j in enumerate(np.unique(y_test2D)):
    plt.scatter(X_test2D[y_test2D == j, 0], X_test2D[y_test2D == j, 1],
                label=j, edgecolor='k')

plt.title("Decision Boundaries after PCA")
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
plt.legend()
plt.show()
```



V. Conclusion

PCA effectively reduces dimensions with minimal loss of variance.

The model performance after PCA is comparable to the full-feature model.

Explained variance shows the first 2 components capture over 95% of the variance.

Dimensionality reduction helps in visualization and training time reduction.

It's important to normalize data before PCA to ensure all features are treated equally.